Year Two Lecture Notes

BSc Computer Science - Durham University

Matthew Emerson

November 2026

Contents

I Artificial Intelligence **1**

1 Module Introduction **1**
1.1 Assessment . . . 1
1.2 Timetable . . . 1

2 Machine Learning **1**
2.1 Introduction to Machine Learning . . . 1
2.2 Machine Learning Workflow and Coding Environment . . . 3
2.3 Data Understanding . . . 3
2.4 Data Preparation Workflow . . . 4
2.5 Data Construction . . . 4
2.6 Data Transformation . . . 5
2.7 Machine Learning Algorithms . . . 6
2.8 Linear Regression . . . 7
2.9 Multivariate Linear Regression . . . 8
2.10 Polynomial Regression . . . 9
2.11 Regression Loss Functions . . . 9

3 Artificial Intelligence Search **10**
3.1 Coding Real-World Problems for Search Tree Expansion . . . 10
3.2 Uninformed Search: BFS, DFS, and Iterative Deepening . . . 11
3.3 General Heuristic Search: Greedy Best-First Search and A* Search . . . 11
3.4 A* Search and Deriving Iterative Deepening A* Search . . . 11
3.5 Heuristic Functions and Their Development . . . 11
3.6 Local Search: Hill-Climbing, Simulated Annealing, and Genetic Algorithms . . . 11
3.7 Swarm Behaviour, How Ants Forage, and a Simple S-ACO algorithm . . . 11
3.8 Applying Different Ant Systems to the TSP . . . 11
3.9 General Versions of Particle Swarm Optimisation and Differential Evolution . . . 11
3.10 Applying Particle Swarm Optimization to the TSP . . . 11

4 Ethics and Bias in AI **11**
4.1 Introduction to AI Ethics and Related Concepts . . . 11
4.2 Societal Impacts of AI . . . 11
4.3 Defining Bias and Fairness in ML . . . 11
4.4 How to Build Better Systems . . . 11
4.5 Value Sensitive Design . . . 11
4.6 Evaluations and Audits . . . 11
4.7 Bias Mitigation Strategies . . . 11
4.8 Transparency and Accountability for Trustworthy Systems . . . 11
4.9 Explainable AI . . . 11
4.10 Future Trends and Ethical Frontiers . . . 11

II Data Science **12**

5 Module Introduction **12**
5.1 Assessment . . . 12
5.2 Timetable . . . 12

6 Probability and Statistics **12**
6.1 Probability and Statistics: Basic Concepts . . . 12
6.2 Dependency and Bayes Theorem . . . 15
6.3 Discrete Probability Distributions . . . 15
6.4 Continuous Probability Distributions . . . 18
6.5 Normal (Gaussian) Distributions . . . 20
6.6 Multiple Random Variables . . . 21
6.7 Correlations . . . 24
6.8 Central Limit Theorem and Sampling . . . 24
6.9 Hypothesis Testing . . . 24
6.10 Data Types and Visualisation . . . 24
6.11 Basic Data Analysis with Python . . . 24

7 Data Cleaning and Analytics **24**
7.1 Data Quality and How to Produce High-Quality Data . . . 24
7.2 Working with Files . . . 25
7.3 Pandas and Numpy . . . 25
7.4 Data Cleaning and Preparation . . . 25
7.5 Python Data Visualisation Libraries . . . 25
7.6 Data Analytics and Reporting . . . 25

8 Computer Graphics **25**

9 Image Processing **26**

III Networks and Systems **27**

10 Module Introduction **27**
10.1 Assessment . . . 27
10.2 Timetable . . . 27

11 Cyber Security **27**
11.1 Introduction, Threat Modelling, and Applied Cryptography . . . 27
11.2 System Security . . . 30
11.3 Authorisation, Authentication, Accountability and Access Control . . . 32
11.4 Network Security . . . 34
11.5 Software Security . . . 37

12 Networks **38**

13 Databases **38**
13.1 Enhanced Entity-Relationship Model . . . 38
13.2 Semistructured Databases (XML) . . . 38
13.3 XML Data Manipulation (XPath and XQuery) . . . 38
13.4 Transactions . . . 38
13.5 Concurrency Control . . . 38
13.6 Distributed Transactions . . . 38
13.7 Distributed Concurrency Control . . . 38

14 Distributed Systems **38**
14.1 Introduction to Distributed Systems . . . 39
14.2 Middleware and RMI Technologies . . . 39
14.3 Message-Oriented Middleware and Web Services . . . 39
14.4 Replication Models . . . 39

14.5 Concurrent Events and Consistency Control . . . 39
14.6 Fault Tolerance . . . 39
14.7 Load Distribution . . . 39

IV Programming Paradigms 40

15 Module Introduction 40
15.1 Assessment . . . 40
15.2 Timetable . . . 40

16 Systems Programming 40
16.1 Version Control and UNIX . . . 40
16.2 The Basics of Shell . . . 41
16.3 The Basics of C . . . 41

17 Functional Programming 43

18 Object Oriented Programming 43
18.1 Java Fundamentals . . . 43
18.2 OOP with Classes and Objects . . . 43
18.3 Encapsulation . . . 43
18.4 Inheritance . . . 43
18.5 Abstraction with Abstract Classes . . . 43
18.6 Abstraction with Interfaces . . . 43
18.7 Polymorphism . . . 43
18.8 Java Collections . . . 43
18.9 Best Practises . . . 43

V Software Engineering 44

19 Module Introduction 44
19.1 Assessment . . . 44
19.2 Timetable . . . 44

20 Project Management 44
20.1 Your Team, Formation, and Leadership . . . 44
20.2 Team Engagement, Negotiation, and Ideation . . . 45
20.3 Planning, Responsibilities, Milestones, and Risks . . . 45

21 Software Engineering 46
21.1 The Software Development Lifecycle . . . 46
21.2 Agile Development in Practice . . . 47
21.3 Project Requirements . . . 48

22 Legal, Social, Ethical, and Professional Issues 50
22.1 Introduction to Software Engineering and Software Engineering Ethics . . . 50

23 Group Project 50

VI Theory of Computation 51

24 Module Introduction **51**
24.1 Assessment . . . 51
24.2 Timetable . . . 51

25 Models of Computation **51**
25.1 Finite-State Automata and Regular Languages . . . 51
25.2 Büchi Automata and Omega-Regular Languages . . . 55
25.3 Linear-Time Logic and Translating Into Büchi Automata . . . 56

26 Algorithms and Complexity **56**
26.1 Asymptotic Complexity and Notation . . . 56
26.2 Greedy Algorithms . . . 56
26.3 Divide-and-Conquer Algorithms . . . 58
26.4 Dynamic Programming Algorithms . . . 62
26.5 Machinings in Graphs . . . 62

VII Appendix **63**

27 Artificial Intelligence **63**
27.1 Module Professors . . . 63
27.2 References . . . 63
27.3 List of Figures . . . 63
27.4 Reading List . . . 63

28 Data Science **64**
28.1 Module Professors . . . 64
28.2 References . . . 64
28.3 List of Figures . . . 64
28.4 Reading List . . . 64

29 Networks and Systems **64**
29.1 Module Professors . . . 64
29.2 References . . . 65
29.3 List of Figures . . . 65
29.4 Reading List . . . 65

30 Programming Paradigms **65**
30.1 Module Professors . . . 65
30.2 References . . . 66
30.3 List of Figures . . . 66
30.4 Reading List . . . 66

31 Software Engineering **66**
31.1 Module Professors . . . 66
31.2 References . . . 67
31.3 List of Figures . . . 67
31.4 Reading List . . . 67

32 Theory of Computation **67**
32.1 Module Professors . . . 67
32.2 References . . . 68
32.3 List of Figures . . . 68
32.4 Recommended Reading . . . 68

Part I
Artificial Intelligence

1 Module Introduction

Artificial Intelligence has the module code COMP2261, and is led by Dr Yang Long.

1.1 Assessment

Title	Opening	Due	Feedback	Professor	Weight
Machine Learning	9th Oct	15th Dec	21st Jan	Dr Yang Long	50%
AI Search	15th Jan	18th Feb	18th Mar	Prof Iain Stewart	25%
Bias in AI	5th Feb	30th Mar	6th May	Dr Shauna Concannon	25%

Table 1: Assessments for Artificial Intelligence.

1.2 Timetable

This module is taught through two one-hour lectures and one two-hour computer class each week as seen below in Table 2.

Day	Time	Class Type	Location	Lecturer
Monday	14:00 - 16:00	Computer Class	TLC025	-
Tuesday	16:00 - 17:00	Lecture	W103	Dr Yang Long
Friday	15:00 - 16:00	Lecture	W103	Dr Yang Long

Table 2: Michaelmas Term Timetable for Artificial Intelligence.

2 Machine Learning

This section is taught by Dr Yang Long. The recommended reading for this topic is:

- Mathematics for Machine Learning (Deisenroth, M.P., Faisal, A.A., and Ong, C.S., 2020).
- Mastering Machine Learning with scikit-learn (Hackeling, G., 2017).

2.1 Introduction to Machine Learning

2.1.1 Module Information

This module is organised around a direct connection between teaching and practical work. The lectures introduce the concepts, mathematics, assumptions, and experimental consequences needed for that week's practical. The practical then produces the evidence used in the corresponding coursework report. This can be seen in Table 3.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

Stage	What you do	What you keep
Lecture	Understand the main question, mechanisms, assumptions, and key equations.	Notes in your own words and questions that still need an answer.
Practical	Implement, test, compare, visualise, and discuss the method.	A notebook that runs from top to bottom with outputs included.
Weekly Report	Select the strongest evidence and explain why the result occured.	A concise report that links theory, method, result, and interpretation.
Portfolio	Revise the nine reports and connect ideas across the module.	A coherent record of technical progress and developing judgement.

Table 3: The learning and coursework model.

For more information about expectations for evidence, see (Long and Durham University, 2026, pp.3-4).

2.1.2 What is Machine Learning?

Definition 2.1.2.1: Arthur Samuel said in 1959 that machine learning is the

> "field of study that gives computers the ability to learn without being explicitly programmed."

The field arose from the question – can a computer learn on its own in order to perform a specific task, rather than relying on a programmer explicitly setting up rules for it? Similarly, Tom Mitchell defined machine learning in 1997 as

> "A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E."

From this, we can define a well-defined learning task as $< T, P, E >$.

There are three typical types of machine learning, these are:

- **Definition 2.1.2.2**: Supervised learning seeks to learn the mapping (rules) between inputs and outputs. Labelled data (correct answers) is provided in past input and output pairs during the learning process to train the model on how it should behave for previously unseen data.
- **Definition 2.1.2.3**: Unsupervised learning seeks to discover the hidden pattern from a set of inputs. Unlabelled data is provided as the past input during the learning process. The program will then attempt to group similar instances into groups.
- **Definition 2.1.2.4**: Reinforcement learning works by providing occasional positive and negative feedback to reinforce behaviours. Good behaviours are rewarded with treats and therefore become more common whereas bad behaviours are punished and therefore become less common. This is a balance between exploration (of unchared territory) and exploitation (of current knowledge).

2.1.3 Differences between AI, ML, and DL

Definition 2.1.3.1: Artificial intelligence is defined as human intelligence being exhibited by machines.

Definition 2.1.3.2: In simple terms, machine learning can be defined as an approach to achieve artificial intelligence.

Definition 2.1.3.3: Deep learning is defined as a technique for achieving machine learning. As such, we can consider AI to be a subset of ML and ML to be a subset of DL.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

2.2 Machine Learning Workflow and Coding Environment

2.2.1 Machine Learning Workflow

Definition 2.2.1.1: To train a model, you use data to create a model and fine-tune it to make better predictions. We can then use this predictive model to make predictions on previously unseen data in order to perform specific tasks.

The machine learning workflow is set out as shown below:

1. **Definition 2.2.1.2**: Project Framing: When setting out to create a model, we must first frame the problem. Here, we set out a clear objective that we want the model to achieve. We then define inputs and outputs and establish success metrics (e.g., prediction accuracy).
2. **Definition 2.2.1.3**: Data Preparation: The next step is to prepare the data. To do that, we identify features (inputs) and a label (output). We then carry out data cleaning on the dataset and split the data into training and test sets.
3. **Definition 2.2.1.4**: Algorithm Selection: Next, we select the algorithm we want to use for the model based on the dataset and the objective.
4. **Definition 2.2.1.5**: Model Training: The training data is used to incrementally improve the model's performance. The progress made in this step is dependant on the quality of the work done in the previous two stages.
5. **Definition 2.2.1.6**: Model Testing: The next step is to take the model and test it against previously unseen data (the test set).
6. **Definition 2.2.1.7**: Hyperparameter Tuning: We can then attempt to improve the training process to further increase the model's accuracy.
7. **Definition 2.2.1.8**: Inference/Prediction: Finally, the model can be deployed for the task outlined in the first step.

2.2.2 Coding Environment

When programming, we traditionally set out the features as variable X and the target label as Y.

2.3 Data Understanding

Definition 2.3.0.1: A sample is the unit for which the model recieves features and, during training, a label.

We then set up a feature matrix X where each row is a different sample and each column is a new feature and y is a matrix where each row is sample's label corresponding to the sample's features in X.

Definition 2.3.0.2: Nearest-centroid classification attempts to classify the samples by setting out one centre for each species and then classifying them based on their nearest centroid. The formula for calculating this is:

$$\mu_c = \left(\frac{1}{N_c}\right) \sum_i : y^{(i)} = cx^{(i)} \tag{1}$$

For each species c, average the feature vectors belonging to that species. Then, subtract the centroid from the new example, square the differences, add them, then take the square root, as shown below.

$$d(x, \mu_c) = ||x = \mu_x||_2 \tag{2}$$

To predict a sample's species, compute the distances from each species' centroid and choose the class with the smallest distance.

$$\hat{y} = arg\ min_c||x - \mu_c||_2 \tag{3}$$

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

2.4 Data Preparation Workflow

Definition 2.4.0.1: To prepare data for machine learning, we use the data preparation workflow which follows the steps as below:

1. Read and inspect the data.
2. Clean and split the data into train and test sets.
3. Fit the baseline and model.
4. Test and compare the model against the baseline.
5. Scale the model and predict unseen data.

Definition 2.4.0.2: We can standardise our dataset using training-set standardisation to reduce the unbalanced impact features with large average Euclidean distances have on predictions. To do this, we use (4).

$$x' = \frac{x - \mu_{train}}{\sigma_{train}} \tag{4}$$

2.5 Data Construction

2.5.1 Analysing the Raw Data

Definition 2.5.1.1: Data can be considered skewed to the right if the majority of the data lies to the left with a long tail to the right. To measure this, we use a measure called skewness which can be calculated with (5)

$$\text{Skewness} = \frac{3 \times (\text{Mean} - \text{Median})}{\text{Standard Deviation}} = \frac{3(\mu - Mdn)}{\sigma} \tag{5}$$

Definition 2.5.1.2: Datasets may sometimes contain capped features. This can be identified by graphing the data. If there is a spike on the maximum-value, then it is likely that the data is capped.

2.5.2 Sampling

When deciding how large of a sample to take from a population, you have to balance the size as a larger sample reduces random sampling noise, but also increases processing time.

Definition 2.5.2.1: The Kolmogorov-Smirnov test compares the distributions of two samples. It returns the maximum vertical distance between two cumulative distribution functions.

$$D = sup_x |F_{sample}(x) - F_{population}(x)| \tag{6}$$

It can then be used to calculate a p-value measuring the evidence against equal distributions. This means that if it is above 5%, then there is insufficient evidence of a difference in the distributions.

Definition 2.5.2.2: Similarly, for categorical data, you can use the Chi-square goodness of fit test which compares the expected and observed counts of each unique value.

$$\chi^2 = \sum_j \frac{(O_j - E_j)^2}{E_j} \tag{7}$$

where O_j is the observed count and E_j is the expected count.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

2.5.3 Building Trustworthy Evaluation Sets

A row should only serve one role. Either as a training set (to fit parameters), a validation set (to compare choices), or as a test set (to get a final accuracy estimate).

Random splitting preserves population proportions only in expectation. A finite test set can contain too many or too few examples from an important region. Rare strata show the largest relative variation.

2.6 Data Transformation

2.6.1 Numerical Transforms

Definition 2.6.1.1: In min-max normalisation, the training minimum becomes 0 and the training maximum becomes 1. Values between them retain their relative position in the observed training range.

$$x' = \frac{x - min_{train}}{max_{train} - min_{train}} \tag{8}$$

Definition 2.6.1.2: In standardisation, values have the training mean subtracted, and then divided by the training standard deviation. The transformed training column has mean near 0 and standard deviation near 1.

$$x' = \frac{x - \mu_{train}}{\sigma_{train}} \tag{9}$$

Definition 2.6.1.3: The estimator interface has three functions: fit (to learn the statistics), transform (to apply statistics), and fit_transform (combines on training). It is also used in imputers, encoders, ColumnTransformer, and Pipeline.

Definition 2.6.1.4: Data leakage occurs when preprocessing learns from rows that should remain unseen. Test-set statistics carry information about the future evaluation distribution back into training.

2.6.2 Skew and Bins

To reduce the impact of skewed columns, they can be log scaled to compress multiplicative differences into smaller additive differences.

$$x' = \log(1 + x) \tag{10}$$

The transformed distribution spreads common values while compressing the tail.

If you use equal-width bins, you may end up with sparse bins if data is skewed. To fix this, we use quartile bins to keep roughly the same number of rows in each bin.

2.6.3 Categories and Missing Values

Definition 2.6.3.1: If a column is unordered, then we can apply a one-hot representation where each unique value in the column becomes a new column with a boolean value of whether it is present. This is done with a one-hot encoder which learns categories during fit and then applies the same column order during transform.

If you have a column with missing data, you can take one of two approaches:

- Delete the incomplete rows - this does remove observations and can impact the data distribution.
- Impute a fitted statistic - preserves row count and creates a value for future rows (normally median).

2.6.4 A Leakage-Free Preprocessing Pipeline

Column-specific preprocessing uses ColumnTransformer to take each column and applies median imputation and standardisation for numeric columns, and categorical encoding for categorical ones.

The end-to-end data flow becomes: taking the raw DataFrame and fit preprocessing, then taking the resulting matrix and training the model.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

2.7 Machine Learning Algorithms

2.7.1 What a Learning Algorithm Contains

Definition 2.7.1.1: A learning algorithm consists of the five parts:

- The representation defines possible predictions.
- The parameters.
- The objective scores each parameter choice.
- The optimiser searches.
- The Hyperparameters control the search rather than the fitted relationship.

Definition 2.7.1.2: A representation states which relationships the model can express. A straight line can change its intercept and slope. It cannot bend, create a threshold, or model separate local patterns.

Definition 2.7.1.3: Parameters determine the fitted line. The algorithm estimates them from the training data. Whereas, the hyperparameters such as learning rate α and iteration count control optimisation. The algorithm does not estimate them through the same update rule.

Definition 2.7.1.4: The objective converts a complete set of predictions into one number. A lower cost means the current parameter values match the training observations more closely under the chosen definition of error.

Definition 2.7.1.5: An optimiser proposes new parameter values. Gradient descent uses local slope information from the objective. Other solvers can reach the same least-squares solution by different routes.

2.7.2 A Straight-Line Model

Definition 2.7.2.1: A simple linear regression model assigns one predicted value to each input x. The intercept sets the prediction at $x = 0$. The slope states how much the prediction changes when x increases by one unit.

$$\hat{y} = \theta_0 + \theta_1 x \tag{11}$$

Definition 2.7.2.2: The hypothesis is a single pair of parameters that produce predictions for every observation.

$$h\theta(x_i) = \theta_0 + \theta_1 x_1 \tag{12}$$

Definition 2.7.2.3: The residual measures the vertical gap between prediction and observation.

$$e_i = \hat{y}_i - y_i \tag{13}$$

Definition 2.7.2.4: The least-squares cost averages squared residuals across all m training observations.

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{j=1}^{m} (h\theta(x_i) - y_i)^2 \tag{14}$$

Definition 2.7.2.5: Each point in parameter space represents one complete line. Moving horizontally changes θ_0 and moving vertically changes θ_1. The height $J(\theta_0, \theta_1)$ records how badly the line fits the data.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

2.7.3 Gradient Descent

Definition 2.7.3.1: The gradient vector is defined as follows, where both partial derivatives aare evaluated using the same current parameter values:

$$\nabla J(\theta_0, \theta_1) = \left[\frac{\delta J}{\delta \theta_0}, \frac{\delta J}{\delta \theta_1}\right]^T \tag{15}$$

Adapting the equation, we can get the gradient with respect to the intercept.

$$\frac{\delta J}{\delta \theta_0} = \frac{1}{M}\sum_i h\theta(x_i) - y_i \tag{16}$$

Adapting the equation, we can get the gradient with respect to the slope.

$$\frac{\delta J}{\delta \theta_1} = \frac{1}{M}\sum_i (h\theta(x_i) - y_i)x_i \tag{17}$$

2.7.4 The Controlled Experiment

When viewing a learning-rate curve we can adapt the hyperparameters based on the emerging patterns.

Pattern	What The Curve Shows	Likely Response
Slow Descent	Cost decreases.	Increase α cautiously.
Smooth Convergence	Cost fails and stabilises.	Retain and verify predictions.
Oscillation	Cost alternates.	Reduce α.
Divergence	Cost grows rapidly.	Reduce α and inspect scaling.

Table 4: Reading the learning-rate curves.

2.8 Linear Regression

2.8.1 Three Routes to the Same Least-Squares Line

The iterative route uses gradient descent. It starts from an initial guess and repeatedly reduces the cost. Its result depends on learning rate, iteration budget, and numerical behaviour.

The direct route uses the closed form to calculate the least-squares parameters from summary quantities. *LinearRegression* packages a robust estimator behind a standard interface.

$$\theta_0 = \frac{\sum_i (x_i - \overline{x})(y_i - \overline{y})}{\sum_i (x_i - \overline{x})^2} \tag{18}$$

2.8.2 Regression Metrics

Mean Square Error:

$$MSE = \frac{1}{m}\sum_i (\hat{y}_i - y_i)^2 \tag{19}$$

Root Mean Squared Error:

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{m}\sum_i (\hat{y}_i - y_i)^2} \tag{20}$$

Mean Absolute Error:

$$MAE = \frac{1}{m}\sum_i |\hat{y}_i - y_i| \tag{21}$$

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

Definition 2.8.2.1: The coefficient of determination compares squared error with the test-set mean baseline.

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \overline{y})^2} \tag{22}$$

2.8.3 Implementation and Optimisation Variants

Definition 2.8.3.1: Batch gradient descent uses every training row for each update. The path is stable, but each update costs a full pass through the data.

Definition 2.8.3.2: Stochastic gradient descent uses one row per update. Mini-Batch gradient descent uses a small group such as 32 rows. Updates become noisier but more frequent.

2.9 Multivariate Linear Regression

2.9.1 From One Feature to Many

In multivariate machine learning models, a prediction adds one weighted contribution per feature. With two features, the fitted object is a plane. Higher dimensions follow the same algebra.

Definition 2.9.1.1: The vector hypothesis is defined as:

$$h\theta(\mathbf{x}) = \theta_0 + \theta_1 x_1 + ... + \theta_n x_n = \theta^T \mathbf{x} \tag{23}$$

Definition 2.9.1.2: The design matrix stacks m examples as rows and n features as columns. The resulting matrix $\mathbf{X}$ lets one matrix multiplication produce every prediction at once. We then prepend a bias column where $x_0 = 1$ for every example which lets the intercept participate in the same dot product as the remaining coefficient. Column 0 contains ones, θ_0 multiplies that column, and there is no separate intercept code.

In this matrix, $\mathbf{X}$ is the design matrix including bias ($m \times (n+1)$), θ is the coefficient vector ($(n+1) \times 1$), $\mathbf{y}$ is the observed targets ($m \times 1$), $\mathbf{X}\theta$ is the predictions ($m \times 1$), and $\mathbf{X}^T(\mathbf{X}\theta - \mathbf{y})$ is the gradient numerator ($(n+1) \times 1$).

Definition 2.9.1.3: The vectorised cost is the residual vector containing one prediction error per row.

$$J(\theta) = \frac{1}{2m}(\mathbf{X}\theta - \mathbf{y})^T(\mathbf{X}\theta - \mathbf{y}) \tag{24}$$

Definition 2.9.1.4: The vectorised graident is the transpose combining the residual evidence for every coefficient at once.

$$\nabla J(\theta) = \frac{1}{m}\mathbf{X}^T(\mathbf{X}\theta - \mathbf{y}) \tag{25}$$

Definition 2.9.1.5: Multivariate gradient descent initialises θ, computes $\mathbf{X}\theta$, forms residuals, computes $\frac{\mathbf{X}^T e}{m}$, and then updates θ.

2.9.2 Feature Scaling and Optimisation Geometry

When feature scales differ greatly, equal changes in different coefficients produce very different movements in prediction space. Gradient descent can zigzag across a narrow valley, slowing progress.

2.9.3 Direct and Iterative Solvers

The normal equation sets the gradient to zero to produce a linear system for the least-squares coefficients.

$$\theta = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y} \tag{26}$$

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

2.10 Polynomial Regression

The prediction curves in x, while every coefficient still enters as a linear multiplier.

$$\hat{y} = \theta_0 + \theta_1 x + \theta_2 x^2 + ... + \theta_d x^d \tag{27}$$

In a known synthetic relationship, thirty training observations sample a smooth signal with Gaussian noise. A separate set of 1,000 observations estimates validation error.

$$y = \sin(2\pi x) + \epsilon \tag{28}$$

2.10.1 Bias

Definition 2.10.1.1: Bias describes systematic error created by restrictive assumptions. Across different training samples, a high-bias model tends to miss the same parts of the true relationship.

Definition 2.10.1.2: Variance describes sensitivity to the particular training sample. A high-variance model changes its fitted curve substantially when the sample changes.

2.11 Regression Loss Functions

2.11.1 What the Model Minimises

Definition 2.11.1.1: A model family states which predictions are possible. The loss states which mistakes matter most. Optimisation then searches for parameters that minimise the average loss.

Definition 2.11.1.2: Positive residuals indicate overpredictions and negative residuals indicate underpredictions, where magnitiude gives size. This can be calculated using:

$$r_i = \hat{y}_i - y_i \tag{29}$$

We can calculate loss using one of the methods specified in Table 5.

Loss	Per-Example Expression	Main Behaviour
Squared	r^2	Quadratic Growth
Absolute	$\|r\|$	Linear Growth
Huber, $\sigma = 1$	$\frac{1}{2}r^2$ near 0, $\|r\| - \frac{1}{2}$ outside.	Smooth and Robust
Log-Cosh	$\log\cosh(r)$	Smooth, nearly linear outside
Quantile, q	$\max(qe, (q-1)e), e = -r$	Asymmetric

Table 5: Methods of calculating loss.

When considering which type of loss to use, it is useful to consider the situation. Some common examples are listed below in Table 6.

Situation	Useful Loss	Reason
Gaussian-like without severe outliers.	Squared	Efficient, smooth, and familiar.
Strong contamination or heavy tails.	Absolute	Limited influence from extreme residuals.
Mostly clean data with some outliers.	Huber/log-cosh	Quadratic near zero and robust in tails.
A conditional percentile is required.	Quantile	Asymmetric loss targets q directly.

Table 6: Common situations and their recommended loss types.

2.11.2 Outliers and Robust Fitting

Definition 2.11.2.1: The squared-loss gradient demonstrates that a large residual produces a proportionally large contribution.

$$\frac{\delta J}{\delta \theta_0} = mean(2r), \frac{\delta J}{\delta \theta_1} = mean(2rx) \tag{30}$$

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

This produces a smooth update where large errors dominate, and that matches MSE fitting.

Definition 2.11.2.2: The absolute-loss gradient removes the residual magnitude from the derivative, making a bounded contribution that is non-differentiable at zero.

$$\frac{\delta J}{\delta \theta_0} = mean(sign(r)), \frac{\delta J}{\delta \theta_1} = mean(sign(r)x) \tag{31}$$

Definition 2.11.2.3: The Huber gradient gives the threshold which clips the influence of large residuals.

$$L'\delta(r) = r \text{ if } |r| \leq \delta, \text{ otherwise } \delta sign(r) \tag{32}$$

When fitting a generic robust line, we use the following steps. When doing this, only loss_grad changes between experiments, with representation and gradient-descent loops staying fixed.

1. Initialise θ.
2. Predict $\hat{y}$.
3. Compute r.
4. Evaluate $L'(r)$.
5. Update both θ values.

2.11.3 Quantile Regression

A mean or median gives one central value. Many decisions need a range that changes with x, especially when outcome variability grows across the input space.

Definition 2.11.3.1: The prediction interval describes where a future individual outcome may fall. It includes irreducible observation noise.

Definition 2.11.3.2: The confidence interval for the mean describes uncertainty in the estimated conditional mean. It narrows as the mean becomes more precisely estimated.

When selecting a regression loss model, you should follow these steps:

1. Identify whether extreme residuals represent noise or real risk.
2. Decide whether the target is a mean, median, or another quantile.
3. Inspect the tail growth of $L(r)$.
4. Inspect whether $L'(r)$ is bounded.
5. Check optimisation smoothness and available solvers.
6. Validate behavious with plots, parameters, and held-out evidence.

3 Artificial Intelligence Search

This section is taught by Professor Iain Stewart. There is no recommended reading for this topic however there may be an additional video to watch after lectures to enable more content to be covered.

Watch the FAQs video.

3.1 Coding Real-World Problems for Search Tree Expansion

Watch ahead of time - this will not be covered in lectures. Week 5 covers no content.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

3.2 Uninformed Search: BFS, DFS, and Iterative Deepening

Watch ahead of time - this will not be covered in lectures. Week 5 covers no content.

3.3 General Heuristic Search: Greedy Best-First Search and A* Search

3.4 A* Search and Deriving Iterative Deepening A* Search

3.5 Heuristic Functions and Their Development

3.6 Local Search: Hill-Climbing, Simulated Annealing, and Genetic Algorithms

3.7 Swarm Behaviour, How Ants Forage, and a Simple S-ACO algorithm

3.8 Applying Different Ant Systems to the TSP

3.9 General Versions of Particle Swarm Optimisation and Differential Evolution

3.10 Applying Particle Swarm Optimization to the TSP

4 Ethics and Bias in AI

This section is taught by Dr Shauna Concannon. The recommended reading is handled on a weekly basis for this topic.

4.1 Introduction to AI Ethics and Related Concepts

4.2 Societal Impacts of AI

4.3 Defining Bias and Fairness in ML

4.4 How to Build Better Systems

4.5 Value Sensitive Design

4.6 Evaluations and Audits

4.7 Bias Mitigation Strategies

4.8 Transparency and Accountability for Trustworthy Systems

4.9 Explainable AI

4.10 Future Trends and Ethical Frontiers

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

Part II
Data Science

5 Module Introduction

Data Science has the module code COMP2271 and is led by Professor Thomas Erlebach.

5.1 Assessment

This module is assessed through two pieces of summative coursework and two final exams, these can be seen in Table 7. Additional formatives are also shown in italics.

Title	Opening	Due	Feedback	Professor	Weight
Data Cleaning & Analytics	Week 11	-	11th Feb	Dr Igor Razgon	25%
Image Processing	15th Jan	2nd Mar	6th May	Dr George Koulieris	25%
Probability and Graphics	May Exam	-	-	Razgon & Koulieris	50%

Table 7: Assessments for Data Science.

Data Cleaning & **Analytics Benchtest Information**: The Data Cleaning & Analytics test in Week 11 is a closed book, multiple choice test, and will be held during practicals (mine is Friday 4pm).

- Closed-book assessment over one hour. Check information closer to the time.
- Comprised of 15 multiple-choice questions, split equally between: basic NumPy, basic Pandas, data cleaning, data analytics, and visualisation.

5.2 Timetable

This module is taught through two one-hour lectures and one two-hour computer class each week as seen below in Table 8.

Day	Time	Class Type	Location	Lecturer
Monday	12:00 - 13:00	Lecture	W103	Dr Igor Razgon
Friday	10:00 - 11:00	Lecture	W103	Dr Igor Razgon
Friday	16:00 - 18:00	Computer Class	CC-0007	-

Table 8: Michaelmas Term Timetable for Data Science.

6 Probability and Statistics

This section is taught by Dr Igor Razgon. The recommended reading for this topic is:

- Probability and Statistics (Degroot, M.H. and Schervish, M.J., 2012).

6.1 Probability and Statistics: Basic Concepts

6.1.1 Counting Theory

Definition 6.1.1.1: The product rule states that, if a procedure can be broken down into a sequence of two tasks with n_1 ways to do the first task and n_2 ways to do the second task, then there are $n_1 n_2$ ways to complete the entire procedure.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

Example 6.1.1.1: John wants to fly to Canada from Germany with a connecting flight through the UK. There are 3 flights from Germany to the UK, and 2 flights from the UK to Canada. How many possible flight combinations are there?

Solution: $3 \times 2 = 6$.

□

Definition 6.1.1.2: The sum rule states that, if you consider two independent sub-tasks (we only have to do one), if the first can be done in n_1 ways, and the second can be done in n_2 ways, then the total number of ways to complete the task is $n_1 + n_2$.

Example 6.1.1.2: Mary wants to choose a student project from three lists: lecturer 1 offers 5 projects, lecturer 2 offers 2 projects, and lecturer 3 offers 10 projects. How many possible options for her?

Solution: $5 + 2 + 10 = 17$.

□

Definition 6.1.1.3: The inclusion-exclusion principle states that, if two tasks can be done at the same time (unlike the sum rule in which they are independent), then there is a possibility of overcounting since some ways may be counted twice. To counter this, we add up the number of ways to do each task, then subtract the number of ways to do both tasks.

$$|A \cup B| = (|A| + |B|) - (|A \cap B|) \tag{33}$$

$$|A \cup B \cup C| = (|A| + |B| + |C|) - (|A \cap B| + |A \cap C| + |B \cap C|) + (|A \cap B \cap C|) \tag{34}$$

Definition 6.1.1.4: An r-permutation of a set of r distinct objects is one ordered arrangement. We denote the number of r-permutations of a set with n elements in as $P(n, r)$.

$$P(n, r) = n \times (n-1) \times (n-2) \times ... \times (n-r+1) = \frac{n!}{(n-r)!} \tag{35}$$

Suppose that a set consists of n objects, of which n_1 are of one type (indistinguishable from one another), n_2 are of a second type, ..., and n_k are of a k_{th} type where $n = n_1 + n_2 + ... + n_k$. The number of permutations of the objects can be calculated with (36).

$$P(n; n_1, n_2, ..., n_k) = \frac{n!}{n_1! n_2! ... n_k!} \tag{36}$$

Definition 6.1.1.5: A r-combination is a counting of distinct objects in an unordered arrangement of length r. The number of r combinations of a set with n distinct elements is given by (37).

$$C(n, r) = \binom{n}{r} = \frac{n!}{(n-r)! \times r!} \tag{37}$$

Example 6.1.1.3: Assume elements A_1, A_2, A_3, A_4 are in a set. We want to choose 2 elements from them.

Solution:

- If the order of elements is important: $P(4, 2) = \frac{4!}{(4-2)!} = 12$.
- If the order of elements is not important: $C(4, 2) = \frac{4!}{(4-2)!2!} = 6$.

□

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

6.1.2 Probability Theory

Definition 6.1.2.1: Probability is the formalisation of uncertain events. It enables one to infer probabilities of interest based on assumptions and observations.

Definition 6.1.2.2: Statistics is the practice of the science of collecting and analysing data. It deals with experimental data and extracting knowledge from it.

Definition 6.1.2.3: An experiment is a procedure that yields one of a given set of possible outcomes.

Definition 6.1.2.4: The sample space of the experiment is the set of possible outcomes.

Definition 6.1.2.5: An event is any subset of the sample space.

Theorem 6.1.2.1: The Theory of Probability states that, given a finite sample space of all outcomes S, assuming that all outcomes are equally likely, for an event set E where $E \subseteq S$, then the probability of occurrence of the event is given by (38).

$$P(E) = \frac{|E|}{|S|} \tag{38}$$

Proof: Proof omitted.

□

Definition 6.1.2.6: The axioms of probability are as follows:

- $P(E)$ is always between 0 and 1: $0 \leq P(E) \leq 1$.
- $P(S) = 1$.
- For disjoint events $E_1, E_2, ..., E_n$ (with no intersection), $P(E_1 \cup E_2 \cup ... \cup E_n) = \sum_{i=1}^{n} P(E_i)$.

Then, from the Axioms of Probability above, we can derive the following:

- **Definition 6.1.2.7**: The complement rule states that $P(\overline{E}) = P(S - E) = P(S) - P(E) = 1 - P(E)$.
- **Definition 6.1.2.8**: The impossible rule states that $P(\emptyset) = 0$.
- **Definition 6.1.2.9**: The general addition rule states that $P(E_1 \cup E_2) = P(E_1) + P(E_2) - P(E_1 \cap E_2)$.
- If $E_1 \subseteq E_2$, then $P(E_1) \leq P(E_2)$.

Example 6.1.2.1: A sequence of 10 bits is randomly generated. What is the probability that at least 1 of these is 0?
Solution: Let E be an event in which all bits are 1.

$$P(E) = \frac{|E|}{|S|} = \frac{1}{2^{10}}$$

Then, by the complement rule, $P(\overline{E}) = P(1 - E)$ where $\overline{E}$ is the event in which at least one bit is not 1.

$$P(\overline{E}) = 1 - \frac{1}{1024} = \frac{1023}{1024}$$

□

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

6.2 Dependency and Bayes Theorem

6.2.1 Conditional Probability

The probability of event A, given that event B happens is given by:

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \tag{39}$$

Therefore, the probability of both A and B occuring can be given by:

$$P(A \cap B) = P(A|B) \times P(B) \tag{40}$$

Definition 6.2.1.1: Events can occur independently. The occurrence of one event is not dependent on the occurrence of another if, and only if:

$$P(A \cap B) = P(A) \times P(B) \tag{41}$$

Definition 6.2.1.2: $P(A \cap B)$ can be presented in two ways, either $P(A|B) \times P(B)$ or $P(B|A) \times P(A)$. Equating these, we can obtain Bayes' Theorem.

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)} \tag{42}$$

This can be rewritten by using variables, where $P(A|B)$ is the posterior, $P(A)$ is the prior, $P(B|A)$ is the likelihood, and $P(B)$ is the evidence.

$$posterior = \frac{likelihood \times prior}{evidence} \tag{43}$$

Suppose that $E_1, E_2, ..., E_n$ are mutually exclusive events such that:

$$\bigcup_{i=1}^{n} E_i = S \tag{44}$$

This means that one of the events **must** occur. Then, in any case, Bayes' Theorem states:

$$P(E_k|A) = \frac{P(E_k) \times P(A|E_k)}{\sum_{j=1}^{n} P(E_j) \times P(A|E_j)} \tag{45}$$

6.3 Discrete Probability Distributions

6.3.1 Random Variables

Definition 6.3.1.1: A random variable is a function that assigns a number to each outcome in the sample space of a random experiment. It is also known as a stochastic variable. They are usually denoted by a capital X or Y.

Definition 6.3.1.2: We say that X is a discrete random variable if X can take a finite number of k values $x_1, .x_2, ..., x_k$ and an infinite sequence of different values $x_1, x_2, ...$ which can be distinguished from one another.

Definition 6.3.1.3: The probability distribution of a random variable X describes the probabilities associated with each possible outcome of X. Let X be a discrete random variable. Suppose that the possible values it can assume are $x_1, x_2, ..., x_n$. Then, these values are associated with probabilities given by

$$P(X = x_k) = f(x_k), k = 1, 2, ..., n \tag{46}$$

This function f is known as the probability mass function (PMF).

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

6.3.2 Distribution Functions

Definition 6.3.2.1: The probability mass function (PMF) is function where the value is a probability and has the following characteristics:

- $f(x_k) \geq 0$.
- $\sum_{k=1}^{n} f(x_k) = 1$.

The mathematical definition is that, for a discrete random variable X with possible values $x_1, x_2, ...$ the probability mass function is a function $f : \{x_1, x_2, ...\} \to [0, 1]$ satisfying $f(x_i) \geq 0$, $\sum_{k=1}^{\infty} f(x_i) = 1$, and $P(X = x_i) = f(x_i)$. They can be calculated by collecting all possible outcomes where $X = x$ and summing their probabilities for all x.

Definition 6.3.2.2: The cumulative distribution function (CDF) or distribution function for a random variable X is defined as

$$F(x) = P(X \leq x) = \sum_{x_i \leq x} P(X = x_i) = \sum_{x_i \leq x} f(x_i), \text{ where } x \in \mathbb{R} \tag{47}$$

Definition 6.3.2.3: The mean (μ) or expected value E of a function is the measure of dispersion or variability in a distribution.

Definition 6.3.2.4: The variance of a function is the measure of dispersion or variability in a distribution.

It is important to consider two laws when discussing random variables:

1. Functions of random variables are also random variables. Let X be a random variable, and $h : \mathbb{R} \to \mathbb{R}$ be an arbitrary function. Then $Y = h(X)$ is also a random variable.
2. **Definition 6.3.2.5**: The law of the unconscious statistician: Let X be a random variable with PMF f and $h : \mathbb{R} \to \mathbb{R}$ be an arbitrary function. Then $Y = h(X)$ is also a random variable. If X is discrete and takes values $\{x_1, x_2, ...\}$ then

$$E[Y] = E[h(x)] = \sum_{i} h(x_i) f(x_i) \tag{48}$$

These two laws result in the conclusion that one can perform arithmetic operations with random variables to get new ones, and the expectation and variance can be computed from them.

- $E(aX + b) = aE[X] + b$.
- $E[h(X)] = \sum_x h(x) f(x)$.
- $V(aX + b) = a^2 V(X)$.
- $V(X) = E(X^2) - [E(X)]^2$.

From this, $E[X]$ is named the first moment, and $E[X]^2$ is the second moment. The variance is also called the central second moment.

We can consider two random variables X and Y to be independent if, for sets A and B, we have

$$P(\{X \in A\} \cap \{Y \in B\}) \equiv P(X \in A, Y \in B) = P(X \in A) P(Y \in B) \tag{49}$$

For independent random variables only, we have

$$V(X_1 + X_2 + ... + X_n) = V(X_1) + V(X_2) + ... + V(X_n) \tag{50}$$

This equation holds for expectation E, whether the variables are independent or not.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

6.3.3 Special Discrete Distributions

Definition 6.3.3.1: The discrete uniform distribution is the simplest discrete random variable. It assumes a finite number of possible values, each with equal probability. A random variable X has a discrete uniform distribution if each of the n values in its range $x_1, x_2, ..., x_n$ has equal probability. Suppose random variable X has uniform distribution. The probability mass function of X is given by $(X \sim Uni(n))$.

$$P(X = k) = f(k) = \frac{1}{n} \tag{51}$$

If we assume X is a discete uniform distribution on the consecutive integers $a, a+1, a+2, ..., b$ for $a < b$, then:

$$\mu = E[X] = \frac{a+b}{2} \tag{52}$$

$$\sigma^2 = Var(X) = \frac{(b-a+1)^2 - 1}{12} \tag{53}$$

Definition 6.3.3.2: The most common discrete distributions are associated with Bernoulli trials. In a random experiment if each outcome consists of only two probabilities, such as in a toss of a coin, then the random experiment is called a Bernoulli trial. In a Bernoulli trial, let the possible events in each outcome be denoted by A and B. Then, $A \cap B = \emptyset$ and $A \cup B = S$. Let the occurrence of A be a success, and the occurrence of B be a fail and let the probability of A be p, then:

$$P(B) = P(\overline{A}) = 1 - P(A) = 1 - p = q \tag{54}$$

Let X be a random variable taking only two possible values $\{0, 1\}$ and let $p = P(X = 1)$. Then X is said to be a Bernoulli random variable with parameter p: $X \sim Ber(p)$, then:

$$E[X] = 0 \times P(X = 0) + 1 \times P(X = 1) = p \tag{55}$$

$$E[X^2] = 0^2 \times P(X = 0) + 1^2 \times P(X = 1) = p \tag{56}$$

$$V(X) = E[X^2] - (E[X])^2 = p - p^2 = p(1-p) \tag{57}$$

Taking (54) and (57), we get that $E[X] = p$ and $V(X) = pq$.

Definition 6.3.3.3: Consider a random experiment consisting of $n \in \mathbb{N}$ independent Bernoulli trials. If the probability of success for each trial equals p, $0 \leq p \leq 1$, and it remains the same throughout the experiment, then we say that the random variable X that equals the number of successful trials is a binomial random variable with parameters n and p. The probability mass function of X is given by $X \sim Bin(n, p)$.

$$P(X = k) = f(k) = \binom{n}{k} p^k (1-p)^{n-k}, k = 0, 1, ..., n \tag{58}$$

The binomial random variable is n independent repetitions of a Bernoulli variable, $X = X_1 + X_2 + ... + X_n$. So we can calculate

$$E[X] = E[X_1 + X_2 + ... + E_n] = E[X_1] + E[X_2] + ... + E[X_n] \tag{59}$$

$$V(X) = V[X_1 + X_2 + ... + X_n] = V(X_1) + V(X_2) + ... + V(X_n) \tag{60}$$

Therefore, $E[X] = np$ and $V(X) = np(1-p)$.

Definition 6.3.3.4: Consider a series of independent Bernoulli trials with constant probability of success p. Let the random variable X denote the number of trials until the first success. Then X is called a geometric random variable with parameter p: $X \sim Geom(p)$.

$$P(X = k) = f(k) = p(1-p)^{k-1}, k = 1, 2, ... \tag{61}$$

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

and $E[X] = \frac{1}{p}$ and $V(X) = \frac{1-p}{p^2}$.

Theorem 6.3.3.1: The geometric distribution has no memory of the past. This is called the memoryless property, meaning that if you have waited k trials already without seeing a success, you must wait m more trials to get success with probability of $p(1-p)^{m-1}$: $P(X = k + m|X > k) = P(X = m)$.

Proof:

$$P(X = k + m|X > k) = \frac{P(\{X = k + m\} \cap \{X > k\})}{P(\{X > k\})} \tag{62}$$

$$\frac{P(X = k + m)}{P(X > k)} = \frac{P(X = k + m)}{1 - P(X \geq k)} \tag{63}$$

$$= \frac{p(1-p)^{k+m-1}}{(1-p)^k} = p(1-p)^{m-1} = P(X = m) \tag{64}$$

□

Definition 6.3.3.5: We can generalise the idea behind the geometric distribution - instead of waiting for the first success, we can wait until r successes. Consider a series of independent Bernoulli trials with constant success probability p. Let the random variable X denote the number of trials until $r \in \mathbb{N}$ successes are attained. Then X is called a negative binomial random variable with parameter p and r: $X \sim NegBin(p, r)$.

$$P(X = k) = f(k) = \begin{cases} 0, \text{ if } k < r \\ \binom{k-1}{r-1} p^r (1-p)^{k-r}, \text{ if } k \geq r \end{cases}, \text{ where } k \in \mathbb{N} \tag{65}$$

Then $E[X] = \frac{r}{p}$ and $V(X) = r\frac{1-p}{p^2}$.

Definition 6.3.3.6: Suppose that you want to model the number of customers entering a store over the period of one day. To do this we make two assumptions: the customers behave independently from one another and the expected number of customers per unit of time is constant λ.

Breaking the day into four-hour intervals gives six pieces ($n = 6$), so the expected number of customers for each interval is $\frac{\lambda}{n} = \frac{\lambda}{6}$. If the intervals are really small, then the number of customers is either 0 or 1 with very high probability. Then the number of customers in each interval is approximately a Bernoulli random variable with $p = \frac{\lambda}{n}$, they are also all independent.

Now look at getting k arrivals in a $Bin(n, p)$, where the number of bins n grows towards $+\infty$. In this case, the probability of a single event will be $p = \frac{\lambda}{n}$.

A Poisson random variable has a distribution that shows how many times an event is likely to happen within a specific period of time. X is called a Poisson random variable with parameter $\lambda > 0$. It is the number of events in a Poisson process, $X \sim Poisson(\lambda)$.

$$P(X = k) = f(k) = \frac{e^{-\lambda}\lambda^k}{k!}, k = 0, 1, 2, ... \tag{66}$$

and $E[X] = \lambda$ and $V(X) = \lambda$.

6.4 Continuous Probability Distributions

6.4.1 Continuous Random Variables

Definition 6.4.1.1: A continuous random variable is a random variable such that the probability of any one specific value is unknown, and thus is treated as zero. Instead, we calculate the probability of the value being in a given interval.

Definition 6.4.1.2: In general, continuous random variables are described by a probability density function. It maps a probability of a range of events from the sample space to a function. The probability density function of a continuous random variable X is denoted f_X or simply f.

$$P(a \leq X \leq b) = \int_a^b f_X(x)dx \tag{67}$$

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

We define the probability density function $f(x)$ for a continuous random variable X such that:

- $f(x) \geq 0, \forall x \in \mathbb{R}$.
- $\int_{-\infty}^{+\infty} f(x)dx = 1$.
- $P(a \leq X \leq b) = \int_a^b f(x)dx, \forall (a,b) | (a,b) \in \mathbb{R}^2, a \leq b$.

Definition 6.4.1.3: Similarly, we can define the continuous cumulative distribution function $F(x) : \mathbb{R} \to [0,1]$ of a continuous random variable X where $x \in \mathbb{R}$ as

$$F(x) = P(X \leq x) = \int_{-\infty}^{x} f(t)dt \tag{68}$$

As in the discrete case, a continuous CDF has the following properties:

- $F(x)$ is non-decreasing: if $x \leq y$ then $F(x) \leq F(y)$.
- $0 \leq F(x) \leq 1$: $\lim_{x \to -\infty} F(x) = 0$, $\lim_{x \to +\infty} F(x) = 1$.
- For $a \leq b$ we have: $P(a \leq X \leq b) = F(b) - F(a)$.

If $F(x)$ is the cumulative distribution function of a continuous random variable X, then its derivative is the probability distribution function $f(x)$.

$$f(x) = F'(x) = \frac{d}{dx}F(x) \tag{69}$$

Example 6.4.1.1: A continuous random variable X has the density function (70). Find the value of the constant c.

$$f(x) = \frac{c}{x^2 + 1} \text{ where } -\infty < x < +\infty \tag{70}$$

Solution: From Definition 6.4.1.3, we know that

$$F(+\infty) = P(X \leq +\infty) = \int_{-\infty}^{+\infty} f(t)dt = 1 \tag{71}$$

Substituting in $f(x)$, we get

$$\int_{-\infty}^{+\infty} \frac{c}{x^2 + 1}dx = [\arctan(x)]_{-\infty}^{+\infty} = 1 \tag{72}$$

$$c \lim_{x \to \infty} [(\arctan(x) - \arctan(-x))]_{-\infty}^{+\infty} = c\left[\left(\frac{\pi}{2}\right) - \left(-\frac{\pi}{2}\right)\right] = 1 \tag{73}$$

This gives us that, $1 = c\pi$, giving $c = \frac{1}{\pi}$.

□

6.4.2 Mean and Variance

Going back to the continuous random variable X, we can define its mean and variance as values to summarise its distribution such that,

$$\mu = E[X] = \int_{-\infty}^{+\infty} xf(x)dx \tag{74}$$

and

$$\sigma^2 = V(X) = E\left[(X - \mu)^2\right] = \int_{-\infty}^{+\infty} (x - \mu)^2 f(x)dx \tag{75}$$

As for discrete random variables, mean and variance have the following properties:

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

- $E[aX + b] = aE[X] + b$.
- $V(aX + b) = a^2V(X)$.
- $\sqrt{V(aX + b)} = |a|\sqrt{V(X)}$.

6.4.3 Specific Continuous Distributions

Definition 6.4.3.1: The continuous uniform distribution sets all samples in a range as having the same probability. Formally, let X be a continuous random variable with density function

$$f(x) = \begin{cases} \frac{1}{b-a}, \text{ if } a \leq x \leq b \\ 0, \text{ otherwise} \end{cases} \tag{76}$$

We have that the mean and variance of this distribution are $E(X) = \frac{b+a}{2}$ and $V(X) = \frac{(b-a)^2}{12}$.

Definition 6.4.3.2: The exponential distribution is used to model the time elapsing between events. The probability of occurrence of an event after a unit of time. It has a parameter λ which is the rate parameter and is the average number of events occuring in a period of one time unit.

Formally, a random variable X is called an exponential random variable with parameter $\lambda > 0$ (denoted by $X \sim Exp(\lambda)$) when the probability density function is

$$f(x) = \begin{cases} \lambda e^{-\lambda x}, \text{ if } x \geq 0 \\ 0, \text{ otherwise} \end{cases} \tag{77}$$

and

$$F(x) = \begin{cases} 1 - e^{-\lambda x}, \text{ if } x \geq 0 \\ 0, \textit{ otherwise} \end{cases} \tag{78}$$

We can then define the mean and variance as $E(X) = \frac{1}{\lambda}$ and $V(X) = \frac{1}{\lambda^2}$.

6.5 Normal (Gaussian) Distributions

6.5.1 Introduction to the Normal Distribution

Definition 6.5.1.1: A normal distribution usually represents the sum of many independent random variables, and almost always looks like a bell curve. These bell curves can have different centres and spreads depending on the mean μ and variance σ^2 of the random variables.

Formally, a random variable X is said to have normal distribution with parameters mean $-\infty < \mu < +\infty$ and variance $\sigma^2, \sigma > 0$, written as $X \sim \mathcal{N}(\mu, \sigma^2)$. The probability density function is defined as:

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \tag{79}$$

with $E(X) = \mu$ and $Var(X) = \sigma^2$.

6.5.2 The Standard Normal Distribution

Definition 6.5.2.1: The standard normal distribution is a special case of the normal distribution defined formally as: A random variable Z with probability density function as follows is called a standard normal random variable:

$$f(z) = \frac{1}{\sqrt{2\pi}} e^{-\frac{z^2}{2}} \tag{80}$$

This is a normal random variable with parameters $\mu = 0$ and $\sigma^2 = 1$. The corresponding cumulative distribution function is denoted as $\Phi(z) = P(Z \leq z)$. This means that any normal distribution X can be converted into a standard normal distribution Z using the formula $Z = \frac{X-\mu}{\sigma}$. This is called the z-score or Z-score, and can be used to compare scores from distributions with different scales, or to determine the relationship between one score and the rest. Before using the normal approximation $P(X \leq x)$ should be changed to $P(X \leq x + 0.5)$.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

6.5.3 Normal Approximations

When you have a binomial distribution where n is large and p is the middle-of-the-road, then the binomial distribution tends to look like a normal distribution.

$$X \sim \text{Bin}(n,p) \Rightarrow P(X \leq x) \approx \Phi\left(\frac{x-np}{\sqrt{npq}}\right) \tag{81}$$

This approximation is generally good for $np > 5$ and $n(1-p) > 5$.

If X is a Poisson random variable with $E(X) = \lambda$ and $V(X) = \lambda$, then

$$X \sim \text{Poisson}(\lambda) \Rightarrow P(X \leq x) \approx \Phi\left(\frac{x-\lambda}{\sqrt{\lambda}}\right) \tag{82}$$

The same continuity correction used for the binomial distribution can also be applied. The approximation is good for $\lambda > 5$.

6.6 Multiple Random Variables

Definition 6.6.0.1: Distributions with two variables are called bivariate distributions, and those with more than two are multivariate distributions.

6.6.1 Bivariate Distributions

Definition 6.6.1.1: The probability mass function of a single discrete random variable X specifies how much probability mass if placed on each possible X value. The joint PMF of two discrete variables X and Y describes how much probability mass is placed on each possible pair of values (x, y).

Definition 6.6.1.2: The notion of join probability distribution can be formally defined as: If X and Y are discrete random variables, the function given by $f(x,y) = P(X = x, Y = y)$ for each pair of values (x, y) within the range of X and Y is called the joint probability distribution (joint probability mass function) of X and Y.

This joint PMF of X and Y inside the range R and S respectively has the following properties:

- $f_{XY}(x,y) \geq 0$.
- $\sum_{x \in R} \sum_{y \in S} f_{XY}(x,y) = 1$.
- $f_{XY}(x,y) = P(X = x, Y = y) \equiv P(\{X = x\} \cup \{Y = y\})$.

Definition 6.6.1.3: Then, we can extend the above definition to the Joint Cumulative Function as follows: If X and Y are discrete random variables, the joint CDF is the function given by:

$$F(x,y) = P(X \leq x, Y \leq y) = \sum_{s \leq x} \sum_{t \leq y} f(s,t) \text{ for } -\infty < x, y < +\infty \tag{83}$$

6.6.2 Splitting the Distribution

Definition 6.6.2.1: Although a bivariate probability distribution describes the joint probability, we may still need to explore X and Y separately. This is done through marginal probability distributions. Basically, the goal is to remove the other variable from the probability distribution.

Let X and Y be two discrete random variables taking values in range R and S respectively with joint probability mass function $f_{XY}(x,y)$. Then, the corresponding marginal probability mass functions are given by:

$$f_X(x) = P(X = x) = \sum_{y \in S} P(X = x, Y = y) = \sum_{y \in S} f_{XY}(x,y) \tag{84}$$

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

$$f_Y(y) = P(Y = y) = \sum_{x \in R} P(X = x, Y = y) = \sum_{x \in R} f_{XY}(x, y) \tag{85}$$

Definition 6.6.2.2: For conditional distributions, let X and Y be two discrete random variables taking values in range R and S respectively with joint probability mass function $f_{XY}(x, y)$. Assume $f_X(x) > 0$ (meaning $X = x$ is possible), then:

$$f_{Y|X=x}(y) = P(Y = y|X = x) = \frac{P(X = x, Y = y)}{P(X = x)} = \frac{f_{XY}(x, y)}{f_X(x)} \tag{86}$$

Similarly, if $f_Y(y) > 0$:

$$f_{X|Y=y}(x) = P(X = x|Y = y) = \frac{P(X = x, Y = y)}{P(Y = y)} = \frac{f_{XY}(x, y)}{f_Y(y)} \tag{87}$$

If X and Y are discrete random variables, then:

- $\sum_{y \in Y} f_{Y|X=x}(y) = 1$.
- For any set B:

$$P(Y \in B|X = x) = \sum_{y \in B} f_{Y|X=x}(y) \tag{88}$$

6.6.3 Continuous Variables

If we consider X taking a value in $[a, b]$ and Y a value in $[c, d]$:

- (X, Y) takes a value in $[a, b] \times [c, d]$.
- If the joint probability density function is $f_{XY}(x, y)$ then:

$$P(a \leq X \leq b, c \leq Y \leq d) = \int_a^b \int_c^d f_{XY}(x, y) dy dx \tag{89}$$

- $f_{XY}(x, y)dxdy$ can be interpreted as the probability of lying in a small rectangle.

Let X and Y be two continuous random variables. The joint PDF of X and Y has the following properties:

- $f_{XY}(x, y) \geq 0$.
- $\int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} f_{XY}(x, y) dy dx = 1$.
- For any set R in the R^2 plane:

$$P((X, Y) \in R) = \int \int_R f_{XY}(x, y) dy dx \tag{90}$$

Let X and Y be two continuous random variables with joint PDF $f_{XY}(x, y)$. Then the corresponding marginal PMFs are given by:

$$f_X(x) = \int f_{XY}(x, y) dy \tag{91}$$

$$f_Y(y) = \int f_{XY}(x, y) dx \tag{92}$$

Let X and Y be two continuous random variables with joint PDF $F_{XY}(x, y)$. Assume $f_X(x) > 0$, then:

$$f_{Y|X=x}(y) = \frac{f_{XY}(x, y)}{f_X(x)} \tag{93}$$

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

similarly, if $f_Y(y) > 0$:

$$f_{X|Y=y}(x) = \frac{f_{XY}(x,y)}{f_Y(y)} \tag{94}$$

as the continuous CDF is actually a proper PDF,

$$\int f_{Y|X=x}(y)dy = 1 \tag{95}$$

and, for any set $B \subseteq \mathbb{R}$:

$$P(Y \in B|X = x) = \int_B f_{Y|X=x}(y)dy \tag{96}$$

6.6.4 Properties of Random Variables

Definition 6.6.4.1: Let X and Y be two random variables (if discrete, with values in R and S; if continuous, with values in $\mathbb{R}$). We say X and Y are independent if one of the following equivalent statements hold:

- $f_X(x)f_Y(y) = f_{XY}(x,y)$ for all $x, y \in R \times S$ (discrete), or $x, y \in \mathbb{R}^2$ (continuous).
- $f_{Y|X=x}(y) = f_Y(y)$ for all $x, y \in R \times S$ (discrete), or $x, y \in \mathbb{R}^2$ (continuous) with $f_X(x) > 0$.
- $f_{X|Y=y}(x) = f_X(x)$ for all $x, y \in R \times S$ (discrete), or $x, y \in \mathbb{R}^2$ (continuous) with $f_Y(y) > 0$.
- $P(X \in A, Y \in B) = P(X \in A)P(Y \in B)$ for any two sets A and B.

Definition 6.6.4.2: Suppose X and Y are discrete random variables taking values in R and S respectively. Let $f_{XY}(x,y)$ be the PMF and $f_X(x)$ and $f_Y(y)$ be the marginal PMFs of X and Y. We can define the means and variances as:

$$E(X) = \mu_X = \sum_{x \in R} x f_X(x) \tag{97}$$

$$E(Y) = \mu_Y = \sum_{y \in S} y f_Y(y) \tag{98}$$

$$V(X) = \sum_{x \in R} x^2 f_X(x) - (E(X))^2 \tag{99}$$

$$V(Y) = \sum_{y \in S} y^2 f_Y(y) - (E(Y))^2 \tag{100}$$

Definition 6.6.4.3: The covariance between two discrete or continuous random variables X and Y is denoted by $cov(X,Y)$ and is given by:

$$cov(X,Y) = \sigma_{XY} = E[XY] - E[X]E[Y] \tag{101}$$

You can also compute covariance by this formula:

$$cov(X,Y) = E[(X - \mu_x)(Y - \mu_y)] \tag{102}$$

Similarly, we can measure the correlation coefficient between two variables using:

$$Corr(X,Y) = \rho_{XY} = \frac{cov(X,Y)}{\sqrt{V(X)V(Y)}} \tag{103}$$

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

6.7 Correlations

6.8 Central Limit Theorem and Sampling

6.9 Hypothesis Testing

6.10 Data Types and Visualisation

6.11 Basic Data Analysis with Python

7 Data Cleaning and Analytics

This section is taught by Dr Igor Razgon. The recommended reading for this topic is:

- Python for Data Analytics (McKinney, 2022).

7.1 Data Quality and How to Produce High-Quality Data

7.1.1 Data Quality

Definition 7.1.1.1: Data cleansing is the process of detecting and correcting (or removing) corrupt or inaccurate records from a record set, table, or database and refers to identifying incomplete, inaccurate, or irrelevant parts of the data and then replacing, modifying, or deleting the dirty or coarse data.

For data to be considered high-quality, it should meet the following criteria:

- **Definition 7.1.1.2**: Validity: The degree to which the measures conform to defined business rules or constraints.
- **Definition 7.1.1.3**: Accuracy: The degree of conformity of a measure to a standard or a true value.
- **Definition 7.1.1.4**: Completeness: The degree to which all required measures are known.
- **Definition 7.1.1.5**: Consistency: The degree to which the data is consistent, within the same data set or across multiple data sets.
- **Definition 7.1.1.6**: Uniformity: The degree to which the data is specified using the same unit of measure.

7.1.2 How to Produce High-Quality Data

Producing high-quality data is an iterative process moving through inspection, cleaning, verifying, and reporting.

Definition 7.1.2.1: Inspection: Detect unexpected, incorrect, and inconsistent data.

- **Definition 7.1.2.2**: Data profiling is the process of viewing the summary statistics of your data to identify standards or patterns in columns of data. Here it is also important to note the data type, any missing values, the number of unique values, the distribution of values, and the relationship between the data and other columns.
- Visualising the data can also be helpful for identifying outliers, as can using statistical methods such as mean, standard deviation, range, and/or quartiles.

Definition 7.1.2.3: Cleaning: Fix or remove the anomalies detected.

- It is important to identify and remove duplicate entries in your data sets.
- When converting the data into a consistent data type, values unable to fit the format should be flagged and converted to NA value.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

- Beyond the data types, the data should be further standardised. This means strings are of a consistent capitalisation, numerical values use the same measurement unit, and dates are in the same format (pick either the UK or US format).
- It can also be useful to scale/transform your data to fit within a certain scale (i.e. 0-100 or 0-1). This can make data easier to plot (e.g., exponential data being logged).
- You can either replace missing values by using a mask to indicate where they are missing, or choosing a sentinel value to replace the missing entries. Conversely, you can also drop the whole record from the data set, or where possible calculate the missing values from other observations.
- **Definition 7.1.2.4**: We define outliers as values which fall 1.5 times the interquartile range from both the first and third quartiles.

Definition 7.1.2.5: Verifying: After cleaning, the results are inspected to verify correctness.

Definition 7.1.2.6: Reporting: A report about the changes made and the quality of the currently stored data is recorded.

7.2 Working with Files

There is no new content here, it is just an introduction to Python's file read/write operations.

7.3 Pandas and Numpy

There is no new content here, it is just an introduction to Pandas and Numpy.

7.4 Data Cleaning and Preparation

7.4.1 Missing Values and Duplicates

See notes from Artificial Intelligence.

7.4.2 Removal of Outliers

Definition 7.4.2.1: When trying to identify outliers, we need to provide precise specifications for a rare occurrence. For this, we generally use one of three interpretations:

- *Threshold Limit*: We set some threshold limit, say 3, and all values that occur fewer than 3 times are considered outliers.
- *Threshold Limit for Clusters*: We classify all the values into several classes and again set a limit, say 3. A value is considered an outlier if its class occurs fewer than 3 times.
- *Quartiles-Based*: We apply a standard statistical methodology involving quartiles and interquartile range.

7.4.3 Splitting and Merging

See notes from Artificial Intelligence.

7.5 Python Data Visualisation Libraries

7.6 Data Analytics and Reporting

8 Computer Graphics

This section is taught by Dr George Alex Koulieris. Check for recommended reading.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

9 Image Processing

This section is taught by Dr George Alex Koulieris. Check for recommended reading.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

Part III
Networks and Systems

10 Module Introduction

Networks and Systems has the module code COMP2211 and is led by Dr Wanqing Tu.

10.1 Assessment

This module is assessed through two pieces of summative coursework and two final exams, these can be seen in Table 9. Additional formatives are also shown in italics.

Title	Opening	Due	Feedback	Professor	Weight
Cyber Security	9th Oct	5th Nov	3rd Dec	Dr Carlton Shepherd	25%
Networks	23rd Oct	21st Jan	18th Feb	Dr Wanqing Tu	25%
Dist. Sys. & Databases	May Exam	-	-	Dr Vasantam	50%

Table 9: Assessments for Networks and Systems.

10.2 Timetable

This module is taught through one two-hour lecture and one two-hour computer class each week as seen below in Table 10.

Day	Time	Class Type	Location	Lecturer
Thursday	09:00 - 11:00	Lecture	CLC202	Dr Carlton Shepherd
Thursday	16:00 - 18:00	Computer Class	CC-0007	-

Table 10: Michaelmas Term Timetable for Networks and Systems.

11 Cyber Security

This section is taught by Dr Carlton Shepherd. The recommended reading for this module is as follows:

- Practical Cryptography for Developers (Nakov, Stefanov and Shideroff, 2018), specifically the section on "Encryption: Symmetric and Asymmetric".

11.1 Introduction, Threat Modelling, and Applied Cryptography

11.1.1 Basic Definitions

Definition 11.1.1.1: A system's security depends on the definition of the adversary: who they are, what they can do (or their capabilities) and their determination (time, money, resources). This is the adversary or threat model.

Definition 11.1.1.2: An asset is something that is worth protecting.

Definition 11.1.1.3: A threat is a person or program that may try and access/steal your assets.

Definition 11.1.1.4: An assumption is something you are relying on to be true/secure by default.

Definition 11.1.1.5: Your goal should be in terms of what needs to be secure, and how long for.

Definition 11.1.1.6: The controls are the things you can do to attempt to protect your assets.

The CIA triad are the three main security properties, these are:

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

- **Definition 11.1.1.7**: Confidentiality: The outcomes of different actions in the system are only visible by authorised subjects.
- **Definition 11.1.1.8**: Integrity: Actions modifying the system can only be performed by authorised subjects.
- **Definition 11.1.1.9**: Availability: Subjects can perform all actions they are authorised for.

In addition to these properties, it is also important to consider additional properties, such as:

- **Definition 11.1.1.10**: Authenticity: An entity is who they claim they claim to be.
- **Definition 11.1.1.11**: Non-Repudiation: An actor cannot easily deny that an action has occurred.
- **Definition 11.1.1.12**: Context-Binding: An action is valid only for a particular period of time, distance, location, and/or environment.

Definition 11.1.1.13: An exploit is a piece of code, a tool, or a technique that takes advantage of a flaw in software, hardware, or a network to cause unintended behaviour.

Definition 11.1.1.14: A vulnerability is a weakness or flaw in a system's design, implementation, operation, or human behaviour that an attacker can exploit to compromise security.

Definition 11.1.1.15: Risk is the probability or liklihood that a threat will exploit a vulnerability and cause harm, loss, or negative consequences to an asset.

11.1.2 STRIDE Modelling

Definition 11.1.2.1: One method of classifying security risks is STRIDE, which was first developed by Microsoft. It is separated as follows:

- **Definition 11.1.2.1**: **S**poofing is the act of using someone else's credentials to perform an operation and is a failure of identification and authentication.
- **Definition 11.1.2.2**: **T**ampering is the malicious modification of data and is a failure of integrity and authenticity.
- **Definition 11.1.2.3**: **R**epudiation is the act of denying having performed an operation and is a failure of accountability and identification.
- **Definition 11.1.2.4**: **I**nformation Dislosure is the exposure of information to unauthorised users and is a failure of confidentiality.
- **Definition 11.1.2.5**: **D**enial of Service is the act of denying service to legitimate users and is a failure of availability.
- **Definition 11.1.2.6**: **E**levation of Privilege is an instance of an unprivileged user gaining privileged access and is a failure of confinement and authentication.

This is a helpful model for structuring the analysis of potential system threats but it is not exhaustive or exclusive.

11.1.3 Applied Cryptography

Definition 11.1.3.1: To encode some given data, it is transformed from one format/domain to another. This is fully reversible, and does not provide any secrecy. The inverse procedure is called decoding.

- **Definition 11.1.3.2**: A subtype of encoding is steganography, where the existence of the secret message is concealed.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

Definition 11.1.3.3: To hash some given data, it is transformed into a (usually) smaller, fixed-size. This is most commonly considered a one-way function. An example of this are SHA algorithms.

- A hashing algorithm works by taking a message m and applying a hashing algorithm $h = H(m)$. h can then be sent securely and the recipient can then take $h' = H(m')$ where $m = m'$, and if $h' = h$, then the message recieved is correct.

Definition 11.1.3.4: Non-secret data (plaintext) can be encrypted into ciphertext using a secret key. The output is $Enc(m, K) = c$. The inverse procedure, decryption, again uses a secret key to transform the ciphertext back to plaintext $Dec(c, K) = m$.

11.1.4 Cipher Examples

Definition 11.1.4.1: A substitution cipher works by taking each letter in the alphabet and replacing it with another one. This was purportedly created and used by Julius Caesar. These are weak though and can be broken by:

- **Definition 11.1.4.1**: A brute-force attack tries every possible key, which for substitution ciphers becomes trivial.
- **Definition 11.1.4.2**: A frequency analysis attack matches the ciphertext to the known frequency of English letters.

Definition 11.1.4.4: Another historical example of a cipher is a Vigenere cipher. This uses a sliding password, such that repeats of the same letter are coded differently. Attacks against this have to guess possible key lengths and then try all possibilities using frequency analysis. This is much harder to break.

Definition 11.1.4.5: Another encryption method is using one time pads. Here, you select a key as long as your message. Data is converted into binary and bitwise XORed with the key. To decode it, you then bitwise XOR the ciphertext with the key. This is considered theoretically secure as long as the key generation process is truly random, the key is never reused, and the key is kept secret.

Definition 11.1.4.6: Stream ciphers use a short secret key and a nonce - a use-once but public value - to generate a long pseudorandom sequence called a keystream. Only the short secret key must be kept private. We then XOR with the message and keystream.

- To encrypt, you take the secret key K and the nonce N and feed them through a keystream generator. The resulting keystream S is then XORed with the plaintext p to produce ciphertext C.
- To decrypt, the recipient takes the same secret key K and nonce N, feeds them through the keystream generator and then XOR the result with the ciphertext C to produce the original plaintext p.

Definition 11.1.4.7: Block ciphers transform a fixed-size block of plaintext into a ciphertext block of the same size, using a secret key. Encryption applies a keyed transformation to the plaintext block, and decryption reverses the transformation using the same key. An example of this is AES encryption. To encrypt longer messages, we use a mode of operation which specifies how to encrypt message blocks in sequence - this usually involves a nonce (initialisation vector).

- **Definition 11.1.4.7**: AES Electronic Code Book (ECB) mode works by taking the plaintext, splitting it into blocks, and then encrypting each of them separately. This is not secure.
- **Definition 11.1.4.8**: AES Cipher Block Chaining (CBC) mode splits the plaintext into blocks and encrypts the first one. The result is then XORed with the second plaintext block before it is encrypted, and so on.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

11.1.5 Public Key Cryptography

Definition 11.1.5.1: Rivest, Shamir, Adelman (RSA) is an algorithm where person A computes the product of two prime numbers $n = p \times q$ and makes the value of n public. Person A then generates a public value e and a private value d using p and q. e can then be used to encrypt messages to send to person A, and d used to decrypt them. This is secure based on the computational difficulty of factorisation, but with the rise of quantum computing, is becoming less secure.

Definition 11.1.5.2: Digital signatures enables others to check that a message was signed using a private key and has not been altered. To sign a message, the sender uses their private key, and then sends the message and signature to the recipient. To verify the signature, the recipient uses the sender's public key and compares the result to the message.

11.2 System Security

11.2.1 Analysing the Security of Systems

Definition 11.2.1.1: A system is a set of components that interact to achieve a purpose. It generally consists of five components:

- **Definition 11.2.1.2**: Subjects are active entities with an identity and capabilities.
- **Definition 11.2.1.3**: Resources are passive entities with an identity and properties.
- **Definition 11.2.1.4**: Actions are interactions between subjects and resources.
- **Definition 11.2.1.5**: The purpose of a system is what it is used for, and so what security must preserve.
- **Definition 11.2.1.6**: A boundary defines what is inside (what you control), and what is outside.

When considering the security of a system, we should consider four questions:

- **What matters?** An asset and a security property (e.g., confidentiality).
- **Who could cause harm?** An adversary with stated capabilities.
- **Why could they succeed?** A weakness and the assumptions it violates.
- **What changes the outcome?** Controls and countermeasures. What is the remaining risk?

Definition 11.2.1.7: A trust boundary is where data or control passes between parties that trust each other differently (e.g., a phone to a ticketing service).

Definition 11.2.1.8: An attack surface is every entry point an adversary can reach.

Definition 11.2.1.9: A trusted computing base (TCB) is what must work correctly for the goal to hold. If any part fails, security fails.

Definition 11.2.1.10: A reference monitor sits between the subjects and objects. It recieves requests for the objects, compares them with the security policy, and if it passes, returns the requested objects. It is important when designing a system that this is always invoked, it is tamper-proof, and that the check is verifiable.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

Actor	Motivation	Typical Capability	Example
Opportunists	Curiosity, notoriety, easy money.	Low: Off-the-shelf tools and mass scanning.	Mirai Botnet (2016).
Hacktivists	Publicity for a cause.	Low-to-Medium: DDoS, leaks, defacement.	Operation Payback (2010).
Organised Criminals	Money.	Medium-to-High: Ransomware, affiliates, access brokers.	Rhysida (2023).
Insiders	Grievance money, or simple mistakes.	High: They already have legitimate access.	Morrisons (2014).
Nation States	Espionage, sabotage, strategic advantage.	Very High: Custom tools, patience, zero-days.	SolarWinds (2020).

Table 11: Potential adversaries to a system.

Example 11.2.1.1: The Hezbollah pager attack was an attack where Hezbollah ordered thousands of new pagers for members across Lebanon and Syria. This consignment was intercepted by agents in a supply chain attack and 10 to 20 grams of military grade explosive was packed into each device. These pagers were then delivered to Hezbollah and handed out to members. The pagers then recieved a code, buzzed, and exploded in pockets, on belts, or in faces of users.

Definition 11.2.1.11: It can also be useful to consider system security from an adversarial perspective. To do this, people commonly use the kill chain model.

- Reconnaissance: Harvesting email addresses, conference information, etc.
- Weaponisation: Coupling an exploit with a backdoor into a deliverable payload.
- Delivery: Delivering a weaponised bundle to the victim via email, USB, etc.
- Exploitation: Exploiting a vulnerability to execute code on the victim's systems.
- Installation: Installing malware on an asset.
- Command and Control (C2): Setting up a control channel for remote manipulation of victims.
- Actions on Objectives: With "hands on keyboard" access, intruders can accomplish their original goals.

Definition 11.2.1.12: Reconnaissance can be split into a few types: passive gathering where attackers have no interaction with the target, active gathering where they do, and open-source intelligence (OSINT) where the attacker analyses data from publically available sources. They do this to identify potential attack vectors.

Example 11.2.1.2: The British Library attack in 2023 can be split into the categories shown in the Kill chain:

- Reconnaissance: Hostile reconnaissance on the 25th-26th October was detected and blocked, but treated as a one-off.
- Delivery and Exploitation: Most likely privileged credentials obtained by phishing or brute force, then used on a remote-access server without MFA. No software exploit was needed.
- Installation and C2: The attackers hijacked the network's own administration tools, and deleted log files to cover their tracks.
- Actions on Objectives: A "historically complex" legacy network let them reach well beyond the entry point and roughly 600 GB was copied. The servers were then encrypted or destroyed.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

11.2.2 Defence

Definition 11.2.2.1: Asymmetry in security is the principle that finding a hole is much easier than securing all possible holes. However, an attack is a sequence of stages that are all opportunities to detect or stop it.

Definition 11.2.2.2: An attack tree is constructed such that: the root is the attackers' goal, the children are the different ways to achieve it, and the leaves are concrete actions. An OR node indicates that any of the children achieves the parent, while an AND node indicates that all children are required to achieve the parent. A set of leaves that satisfies the root is a complete attack.

When designing a system, it is important to consider the following principles: minimise the attack surface, split your defence across many layers, assign users as few privileges as necessary, deny requests as a default reponse unless explicitly allowed, attackers will always take the weakest link, and you should always assume there will be a breach, so monitor and plan for recovery.

11.3 Authorisation, Authentication, Accountability and Access Control

Definition 11.3.0.1: Identification identifies who someone claims to be.

Definition 11.3.0.2: Authentication tests whether a person can prove their identity.

Definition 11.3.0.3: Authorisation defines what a user is allowed to do.

Definition 11.3.0.4: Access control ensures that a user's authorisation is enforced on every request.

Definition 11.3.0.5: Accountability ensures that actions can be traced back to the user.

11.3.1 Authentication

Definition 11.3.1.1: The authentication lifecycle is separated into five steps. These are:

- **Enrolment** binds an identity to a credential. This can go wrong if an attacker registers as you.
- **Authentication** requires the user to present their credentials for the verifier to check. This can go wrong after brute-force, phishing, or stolen password attacks.
- A **session** sets up a token to stand in for you on every later request. This goes wrong if a stolen cookie skips the login entirely.
- **Recovery** regains access when a credential is lost. This can go wrong if there are guessable security questions, or by exploiting the help-desk.
- **Revocation** removes access when people leave, or when credentials leak. This goes wrong if old codes still work.

Definition 11.3.1.2: Multi-factor authentication can make a system more secure. The factors you can use are: something you know; something you have; and something you are.

Definition 11.3.1.3: No authenticator is perfect, we can judge their accuracy by measuring their false acceptance rate and false rejection rate. Generally, lowering one raises the other. For example, touch ID has a FAR of $1 : 50,000$ and face ID has a FAR of $1 : 1,000,000$.

Definition 11.3.1.4: Each authentication attempt gives a similarity score, not a boolean value. The threshold (visualised in Figure 1[1]) turns it into a decision. If we raise the threshold, then there will be fewer false accepts, but more false rejects, and vice-versa.

[1]Image taken from Deparment of Computer Science (2026).

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

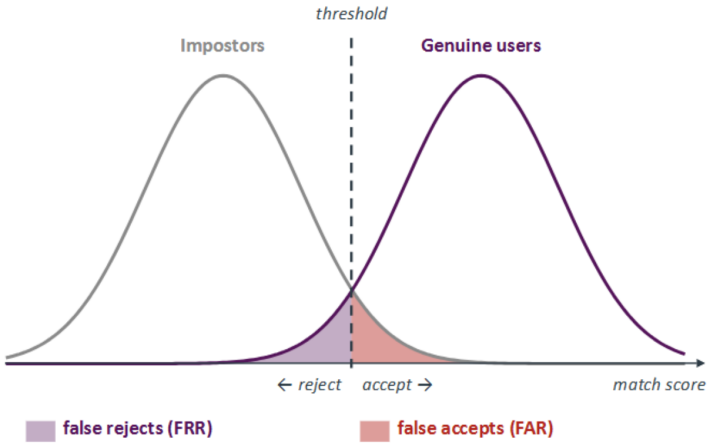


Figure 1: Threshold Trade-off.

Definition 11.3.1.5: The equal error rate is where $FAR = FRR$. This is good for comparing systems, but is rarely the right operating point. The right setting depends on what each error costs.

Passwords persist because they are free and work anywhere. However, because people choose them, reuse them, and/or type them anywhere, they can become insecure. Some common attacks against passwords are in Table 12.

Attack	How It Works	Main Defence
Online Guessing	Try common passwords against the login page.	Rate limiting, blocklists of common passwords.
Credential Stuffing	Replay uername-password pairs leaked from other sites.	MFA, checks against breached passwords.
Phishing	A look-alike site collects whatever the user types.	Phishing-resistant authentication (passkeys).
Infostealer Malware	Malware copies saved passwords and session cookies.	Endpoint protection, device-bound credentials.
Offline Cracking	Steal the password database, guess at leisure, no rate limit.	Salted, slow password hashing.

Table 12: Common attacks against passwords.

When hashing passwords, you should aim to use a slow hash function so that you can reduce the number of guesses per second, and therefore increase the time required to try all possible passwords. For example, SHA-256 allows for 28 billion guesses per second, and so passwords can be guessed in a maximum of 2 minutes, whereas bcrypt only alows 9.5 thousand guesses per second, therefore increasing the time required to around 9 years.

Definition 11.3.1.6: Entropy measures uncertainty in bits. A secret chosen uniformly from S possibilities has $H = \log_2 S$ bits.

$$H(N, L) = \log_2(N^L) = L \log_2 N \tag{104}$$

where N is the size of the alphabet, L is the length.

Definition 11.3.1.7: In relay attacks, the attacker never breaks the cryptography. Instead, they forward the challenge and the response so both ends belive they are closed. To defend against this, you can introduce distance bounding where you time the challenge-response, or to require a user action.

Multi-factor authentication can be defeated if attackers use one of the techniques in Table 13.

Technique	How It Works
Push Fatigue	Spam approval requests until the user taps “approve”.
Real-Time Phishing Proxy	A fake site relays password and code to the real one, then keeps the session cookie.
SIM Swap	Persuade the mobile operator to move your number to the attacker’s sim.
Help-Desk Social Engineering	Phone IT support as the victim and have MFA reset.
Session Theft	Steal the cookie after login, MFA has already happened.

Table 13: Common attacks against MFA.

11.3.2 Authorisation and Access Control

Definition 11.3.2.1: Access control can be implemented using one of the following approaches:

- Discetionary (DAC) where the owner decides who gets access. Examples of this are Unix permissions, Windows ACLs, or sharing a Google Doc. It is flexible, but anything running as you, including malware, can give your data away.
- Mandatory (MAC) where a system-wide policy decides and users cannot override it, even for files they create. Examples include government classifications or SELinux. It is rigid byt contains Trojan horses and careless users.
- Caoabilities where unforgeable tokens are held by subjects. It is good for delegation or distributed systems, but can lead to issues with revocation.
- Role-Based (RBAC) where permissions attach to roles which are assigned to users. Joins, moves, and leaves change one assignment, rather than hundreds of permissions.
- Attribute-Based (ABAC) where you decide the access from attributes of the subject, the object, the action, and the environment. This is good because it is fine-grained and context aware, but can lead to compications when trying to work out who can access a given file.

An important principle is that users with a set access level cannot read up (you can only read an object if your clearance is $\geq$ its classification), and cannot write down (write an object only if its classification $\geq$ your clearance.)

Access control can fail when: the user is logged in, but nobody checks that this record is theirs; admin pages are protected only by hiding the link from the menu; or a role/price is in a hidden field, cookie, or JavaScript.

11.3.3 Accountability

When maintaining a system you should have methods for automatically logging: logins (successes and failures); MFA changes, password resets and recovery; role/permission changes; access to sensitive objects and denied requests; and administrative actions.

These logs should be protected by placing them off the host and making them append-only. They should also either use synchronised clocks, or there is no timeline, and should have an alert on patterns. They should also not allow any shared accounts.

11.4 Network Security

When using a network, each hop gives an opportunity for an attacker to read (confidentiality issue), change (integrity issue), drop or flood (availability issue), or fake (authenticity) your traffic.

These attackers can take one of three forms:

- **Definition 11.4.0.1**: An off-path attacker is not on your path. They can send packets with any source address, but cannot see the replies.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

- **Definition 11.4.0.2**: A on-path attacker sees every packet. They can read, modify, drop, inject, and/or replay.
- **Definition 11.4.0.3**: A volumetric attacker needs no cleverness, it just sends more traffic than a target system can handle.

11.4.1 Protocols

Each layer of the network stack wraps the one above it in its own header (e.g., data → TCP segment → IP packet → WiFi or Ethernet frame). A header is just bytes that the sender wrote. The original protocols check for accidents (checksums), but these are not necessarily for security. The security was instead added later at different layers:

- The application layer: DNSSEC (signed DNS answers) and HTTPS.
- The transport layer: TLS.
- The network layer: IPsec, WireGuard (VPNs).
- The link layer: WPA2/WPA3 (WiFi).

UDP uses a "fire and forget" approach to communication. It has no connection, no ordering, and no retransmission. Examples of its implementations include in DNS, NTP, video calls, and games.

Conversely, TCP is a reliable byte stream. It uses handshakes, sequence numbers, acknowledgements, and retransmission. It is used by the web, email, and SSH.

Definition 11.4.1.1: The handshake system that TCP uses is called the three-way handshake. In practice, it uses the following steps:

1. The client sends a *synchronise* SYN request with a sequence number x.
2. The server responds with a *synchronise acknowledgement* $SYN-ACK$ with a sequence number y and an acknowledgement number $x+1$. An acknowledgement ACK says "I have everything up to here", so each side proves it saw the other's number. Each side counts its bytes from its own random start.
3. After the SYN, the system must remember the client. It does this by keeping one slot in its queue for half-open connections.
4. Once the client recieves the $SYN-ACK$ they send one more ACK with acknowledgement number $y+1$ before sending the data. This means that if y is guessable, an attacker that never sees the $SYN-ACK$ can still send the right ACK.

Definition 11.4.1.2: DNS records are stored in one of a few set types which provide various information about a target host. The common types are:

- Address (A) records which give the IP address for the host.
- AAAA records which provide an IPv6 address for the host.
- Cannonical name (CNAME) records which map one name to another and is used for redirection.
- Mail exchange (MX) records which the server uses for sending/recieving emails.
- Nameserver (NS) records which indicate the authoritative DNS server for this domain.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

11.4.2 Flooding Attacks

Definition 11.4.2.1: Denial-of-service attacks max out the bandwidth of a service using UDP floods, reflection, and amplication. The attack also manipulates the states by maliciously filling up look-up tables using DNS cache poisoning and SYN floods. It achieves the denial of service by making each request expensive for the application using HTTP floods on search/login pages. The attack becomes distributed (DDoS) when thousands/millions of sources, either hijacked devices or innocent servers, are used to make the requests.

Definition 11.4.2.2: A DNS cache poisoning attack is used by an off-path attacker who waits for a resolver to make a request to a server, before then flooding the resolver with forged answers before the real one arrives.

Definition 11.4.2.3: The DNSSEC system uses the .root's $DNSKEY$ (the root key, KSK-2024) which is used to vouch for .com which has a DS from .root. Then, example.com uses the DS from .com which authenticates it. This system provides origin authentication and upkeeps the integrity of answers. However it does not provide confidentiality, availability, or protection for the last hop.

Definition 11.4.2.4: A *SYN flood attack* floods a server with SYN requests with random, spoofed sources. These are cheap to send as they consist of one packet and don't require a reply, but they are costly to recieve as each SYN holds a queue slot until it times out. Once the queue fills up, genuine users will be turned away.

This can be prevented by remembering nothing after a request. Instead, the system designs its sequence number y to be 32-bits, split into t which is 5 bits, MSS which is 3 bits, and then MAC which is constructed from a secret, the client IP, the client port, the server IP, the server port, and t. This means that when a SYN arrives, the server replies with y and stores nothing. A real client would then return $ACK = y + 1$. The server can then reconstruct the MAC, and if they match it will establish the connection.

11.4.3 Cryptography on the Wire

On a network, cryptography is applied in many layers: WiFi (WPA2/WPA3) acts between you and the router; a VPN tunnel acts between you and a VPN server; TLS (HTTPS) works all the way between you and the server; and DNSSEC works between you and the ISP.

Definition 11.4.3.1: TLS acts as the security layer between HTTP and TCP/IP. It ensures confidentiality by encrypting traffic, provides integrity by detecting any changes in transit (AEAD), and authenticates the server by adding a certificate to prove that you reached the name you asked for. It also uses a handshake algorithm, called the TLS 1.3 handshake:

1. The browser sends a *ClientHello* with a key share and cipher suites.
2. The server then responds with *ServerHello* with a key share.
3. These two shares are used to create a shared key where the key parameters are unique to the session and deleted post-use. Stealing the server's key later does not decrypt today's traffic.
4. The server then uses this shared key to send *Certificate*, *CertificateVerify*, and *Finished*.
5. The browser then also sends *Finished*, followed by the application data.

TLS fails in practice when it is implemented wrongly, if the certificate authorities cannot be trusted, if the protocols you are using are downgraded/stripped en-route, or through exploiting the users of the system.

11.4.4 Virtual Private Networks (VPNs), Firewalls, and Segmentation

Definition 11.4.4.1: Virtual private networks (VPNs) are effectively "tunnels through the Internet". They are generally one of:

- IPsec + IKEv2 which is the standard for site-to-site communication and is built into most operating systems.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

- TLS-based which is used by OpenVPN and many other vendors' SSL VPNs.
- WireGuard which is a bespoke protocol which has been used in Linux since 2020.

Definition 11.4.4.2: Firewalls are a reference monitor for the network. It checks every incoming packet against the network policy and refuses any that don't meet the standard. It can consist of any from:

- A packet filter which is stateless and checks each packet's addresses, ports, and protocol on its own.
- Connnection tracking which is stateful and remembers connections to allow replies to existing connections.
- The application layer which uses a proxy, NGFW, and WAF to read the content of packets, including HTTP requests, DNS queries, and users.

11.5 Software Security

11.5.1 Software Attackers

Definition 11.5.1.1: A remote software attacker sends input that controls any bytes the program reads: packets, form fields, files, headers, and/or log lines. Examples of this include Heartbleed, SQL injection, and LogShell.

Definition 11.5.1.2: A local software attacker is on the machine and already runs as an ordinary user or app, and wants more privilege. Examples of this include PwnKit, kernel bugs, and phone jailbreaks.

Definition 11.5.1.3: An upstream software attacker is in the supply chain and gets code into your program before you run it, e.g., a library, build server, or update. Examples of this include XZ Utils, SolarWinds, and npm worms.

11.5.2 Memory Safety

Definition 11.5.2.1: A buffer overflow attack is where an attacker sends a too large input, running straight over the return address. This means that the attacker can choose the bytes and point the new return address wherever they like. When the input function finishes, the attacker, rather than the programmer, decides what runs next.

Some other common main memory bugs are:

- *Out-of-bounds read*, where the input reads past the end of a buffer, such as with Heartbleed or Crowdstrike.
- *Integer overflow*, where a number wraps around, often a size, such as with Stagefright.
- *Format string*, where an input becomes printf's instructions, such as the iPhone Wi-Fi name.
- *Use after free*, where memory is used after it was given back, such as with many browser zero-days.

Developers can mitigate the risks of these attacks by implementing:

- *Stack canary*, where a random value below the return address is checked before returning. If it has changed, then the program stops.
- *No-execute (NX)*, where data areas are marked as not runnable, so smuggled bytess will not run.
- *ASLR*, where everything loads at random places each run.
- *Control-flow checks*, such as in newer CPUs, which keep a protected copy of each return address.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

11.5.3 Injection

To prevent SQL injections, code and data should be kept apart. The query's structure should be fixed first and then the values should travel separately and are only ever data. Similarly, least privilege and a web application firewall should be implemented to restrict what the web app is allowed to run.

11.5.4 Finding and Fixing

A vulnerability falls into one of 5 lifecycle stages: the bug is first written, the bug is found, the vendor is told about the bug, a patch is released, the patch is installed.

11.5.5 Digital Forensics

When examining an incident, you should never change data that may be relied on in court. If you must touch originals, be competent, and able to explain what your actions did. Keep an audit trail – an independent examiner should get the same result. The person in charge is responsible for following the law and these principles.

Similarly, you should consider the order of volatility when collecting data about an incident. CPU registers, cache, and memory may be replaced early, whereas temporary files, disk, remote logs/monitoring, or backups/archives will not.

Evidence of an incident can be: logs about who connected, when, and what they sent; timestamps for when data was created, modified, accessed; deleted files that have not yet been overwritten; and metadata such as authors and edit history.

12 Networks

This section is taught by Dr Wanqing Tu. Check for recommended reading.

13 Databases

This section is taught by Dr Vasantam. The recommended reading for this topic is:

- Connolly, T.M. and Begg, C.E. (2015). Database Systems: A practical approach to design, implementation, and management. 6th edn. Harlow, Essex, England: Pearson Education Limited.

13.1 Enhanced Entity-Relationship Model

13.2 Semistructured Databases (XML)

13.3 XML Data Manipulation (XPath and XQuery)

13.4 Transactions

13.5 Concurrency Control

13.6 Distributed Transactions

13.7 Distributed Concurrency Control

14 Distributed Systems

This section is taught by Dr Vasantam. The recommended reading for this topic is:

- Van Steen, M. and Tanenbaum, A.S. (2025). Distributed Systems. 4th edn. [online] Self-Published. Available at: https://www.distributed-systems.net/index.php/books/ds4/ [Accessed 5 Oct. 2026].

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

14.1 Introduction to Distributed Systems

14.2 Middleware and RMI Technologies

14.3 Message-Oriented Middleware and Web Services

14.4 Replication Models

14.5 Concurrent Events and Consistency Control

14.6 Fault Tolerance

14.7 Load Distribution

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

Part IV
Programming Paradigms

15 Module Introduction

Programming Paradigms has the module code COMP2221 and is led by Dr Stuart James.

15.1 Assessment

This module is assessed through one piece of summative coursework and two final exams, these can be seen in Table 14. Additional formatives are also shown in italics.

Title	Opening	Due	Feedback	Professor	Weight
Systems Programming	*12th Oct*	*12th Nov*	*10th Dec*	*Dr Stuart James*	-
Systems Programming	23rd Oct	10th Dec	21st Jan	Dr Stuart James	50%
Object Oriented & Functional	May Exam	-	-	Ivkovic & Moses	50%

Table 14: Assessments for Programming Paradigms.

15.2 Timetable

This module is taught through two one-hour lectures and one two-hour computer class each week as seen below in Table 15.

Day	Time	Class Type	Location	Lecturer
Monday	11:00 - 12:00	Lecture	CLC202	Dr Stuart James
Monday	16:00 - 18:00	Computer Class	CB-LG001	n/a
Friday	14:00 - 15:00	Lecture	CLC202	Dr Stuart James

Table 15: Michaelmas Term Timetable for Programming Paradigms.

16 Systems Programming

This section is taught by Dr Stuart James. The reading list for this module is as follows:

- Bryant, R. and O'Hallaron, D. (2015). Computer systems: A Programmer's Perspective. 3rd edn. Pearson Education, Limited.

I have not made notes on the majority of this module as most of it is simply learning C. Where lecture contents differ from this, I have made notes.

Definition 16.0.0.1: Systems programming involves the development of individual pieces of software that allow the entire system to function as a single unit.

16.1 Version Control and UNIX

16.1.1 Introduction to UNIX

Files are stored in a hierarchical structure, allowing for grouping. To navigate around these files, we can use UNIX navigation commands as seen in Table 16.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

Action	Command
List the contents of the current folder.	*ls*
Change folder/directory.	*cd*
Make new folder/directory.	*mkdir*
Move a file/folder.	*mv*
Copy a file/folder.	*cp*
Delete a file or, by adding $-r$, a directory.	*rm*
Check file/folder size.	*du*
List all files.	*find*

Table 16: UNIX file navigation commands.

16.2 The Basics of Shell

All shell files are saved with a *.sh* extension and must begin with #!*/bin/bash*.

16.2.1 Basic Commands

```
#!/bin/bash
for f in *
do
    #something in here
    echo $f
done
```

Algorithm 1: For loop example.

```
./myscript.sh 10 2

#!/bin/bash
if [$1 -lt $2]
then
    echo "yes" $1 "is less than" $2
else
    echo "no it isn't"
fi
```

Algorithm 2: If statement example (compares 10 and 2).

16.3 The Basics of C

16.3.1 Basic File Setup

Definition 16.3.1.1: Pre-processor directives are lines which start with a # and are commands to the C pre-processor.

- If the directive is #include, and <> are used, then */usr/include* is prioritised in UNIX by convention, whereas if "" are used, then the current directory is prioritised.
- **Definition 16.3.1.1**: If the directive is #define, then this is a definition macro. These are used to provide definitions in code.

- **Definition 16.3.1.2**: You can also use directives for debugging without generating overheads, these are called conditionals. They are as shown in Table 17.

Conditional	Function
#ifdef VAR	Tests if VAR is #defined.
#ifndef VAR	Tests if VAR is not #defined.
#else	Runs if the original condition evaluates as false.
#endif	Ends the if testing condition.

Table 17: Different types of conditionals.

You can also put debugging code in the preamble, as seen in Algorithm 3.

```
#define MY_DEBUG // Define an identifier.

#ifdef MY_DEBUG
    assert(i > 0);
    printf("i is %d \n", i);
#endif
```

Algorithm 3: Debugging Code Example.

Definition 16.3.1.4: You can also use #define to define parameterised macros. These act like functions, and an example can be seen in Algorithm 4.

```
#define ADD(a,b) a + b
```

Algorithm 4: Parameterised Macro Example.

Definition 16.3.1.5: The main() function is the entry function for C programs. You can only have one of these.

16.3.2 Printf Parameters

Value	Resulting Format
$\%d$	Signed decimal (int).
$\%u$	Unsigned decimal.
$\%o, \%x$	Octal and hexidecimal.
$\%l$	Long.
$\%f$	Floating point, so $\%1.2f$ will give 3.14.
$\%e$	Floating point (exponent form).
$\%c, \%s$	Character and string.

Table 18: First parameter of printf.

16.3.3 Variables

Each variable in C has one of the following four storage types.

- **Definition 16.3.3.1**: extern: When a variable is defined, it is allocated storage. When a variable is declared, the compiler knows a variable of a given type exists. Top-level variables default to extern storage class. They last for the lifetime and scope of the whole program.

- **Definition 16.3.3.2**: static: Static variables have the same lifetime as the program, static global variables have file scope, and static local variables have function scope. A static variable inside a function keeps its value between invocations.
- **Definition 16.3.3.3**: auto: Auto variables have the same lifetime as the function where they are defined. They have function scope and function lifetime.
- **Definition 16.3.3.4**: register: Suggests that a variable should be stored in a register rather than in main memory. It cannot use the address of (&) operator.

17 Functional Programming

This section is taught by Dr Billy Moses Jr.. Check for recommended reading.

18 Object Oriented Programming

This section is taught by Dr Mladen Ivkovic. Check for recommended reading.

18.1 Java Fundamentals

18.2 OOP with Classes and Objects

18.3 Encapsulation

18.4 Inheritance

18.5 Abstraction with Abstract Classes

18.6 Abstraction with Interfaces

18.7 Polymorphism

18.8 Java Collections

18.9 Best Practises

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

Part V
Software Engineering

19 Module Introduction

Software Engineering has the module code COMP2281 and is led by Dr Craig Stewart.

19.1 Assessment

This module is assessed through a group project, the deadlines for this can be seen in Table 19. Additional formatives are also shown in italics.

Title	Opening	Due	Feedback	Professor	Weight
Peer Evaluation 1	16th Oct	19th Nov	10th Dec	Dr Craig Stewart	1.67%
Requirement Documentation	16th Oct	19th Nov	17th Dec	Dr Craig Stewart	15%
Peer Evaluation 2	16th Oct	11th Feb	11th Mar	Dr Craig Stewart	1.67%
Product Presentation	Week 16	-	18th Mar	Dr Craig Stewart	15%
Technical Report	5th Feb	18th Mar	6th May	Dr Craig Stewart	30%
Peer Evaluation 3	5th Feb	18th Mar	6th May	Dr Craig Stewart	1.67%
Git Commit	5th Feb	29th Apr	1st Jul	Dr Craig Stewart	5%
Project Handover	5th Feb	29th Apr	1st Jul	Dr Craig Stewart	10%
Reflective Experience Report	5th Feb	6th May	1st Jul	Dr Craig Stewart	20%

Table 19: Assessments for Software Engineering.

19.2 Timetable

This module is taught through two one-hour lectures and one two-hour computer class each week as seen below in Table 20.

Day	Time	Class Type	Location	Lecturer
Tuesday	09:00 - 10:00	Lecture	TLC033	Dr Nelly Bencomo
Wednesday	11:00 - 12:00	Lecture	TLC033	Dr Nelly Bencomo
Thursday	11:00 - 13:00	Practical	CC-0007	Dr Gagangeet Aujla

Table 20: Michaelmas Term Timetable for Software Engineering.

These lectures are taught jointly by Dr Craig Stewart and Dr Nelly Bencomo.

20 Project Management

20.1 Your Team, Formation, and Leadership

This lecture was taught by Dr Nelly Bencomo.

Belbin (https://www.belbin.com/about/belbin-team-roles) defines nine roles someone can take in a team. These are:

- The Innovator who solves problems as the "ideas guy" of the group.
- The Resource Investigator who contacts external people/companies to bring additional resources to the group.
- The Coordinator who guides and controls the team. They know their team's abilities well.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

- The Shaper who is a demanding and confrontational leader who pushes their members to excel.
- The Monitor/Evaluator who analyses proposed solutions and choices.
- The Team Worker who facilitates team functions and mediates issues.
- The Implementor who implements the agreed upon plans.
- The Completer/Finisher who focuses on details and meeting deadlines.

After assigning roles and completing skill evaluation tests, the team can then create a CPD plan which identifies existing skills, areas for development, and a plan for doing that.

Maslow, A. (2016) splits competency into four categories: unconscious incompetence, conscious incompetence, unconscious competence, and finally conscious competence.

20.2 Team Engagement, Negotiation, and Ideation

This lecture was taught by Dr Nelly Bencomo.

When negotiating, you are more likely to be successful if you have the following factors: legitimacy, confidence, courtesy, adaptation to the other party, rapport, incentives/trade-offs, and research into the bigger picture.

20.3 Planning, Responsibilities, Milestones, and Risks

This lecture was taught by Dr Nelly Bencomo.

A simple task becomes a project when it produces a unique outcome, a finite timeline, interdependant work, and some level of uncertainty.

The project triangle states that there are always trade-offs between scope, resources, and time. This is because quality is never optional.

20.3.1 Planning

Plan outcomes before tasks:

- Goal: What useful result will the client recieve?
- Deliverable: What evidence or artefact demonstrates it?
- Milestone: When will we check progress?

Never adjust deadlines - its easier to instead negotiate scope.

RACI: A responsibilty assignment matrix for making decisions and producing work.

- Responsible: The person that does the work.
- Accountable: The person that owns the outcome/approval.
- Consulted: The person that provides input before action.
- Informed: The person who recieves progress updates.

Activity	Lead	Developer	Tester	Client
Agree Prject Scope	R	C	C	A
Implement a Feature	A	R	C	I
Test a Feature	A	C	R	I
Demonstrate Prototype	A	R	R	C

Table 21: RACI assignment matrix

Project Management Tools: Trello (time management), GitHub Projects, Jira (scrum, sprints, backlogs), and/or Microsoft Planner (team tasks and deadlines).

Assigning priority level, using impact (1-5: negligible, minor, moderate, major, crucial) and risk (1-5: negligible, low, moderate, high, massive). Then set priority bands (e.g., 1-5 is low, 6-14 is medium, 15-25 is high).

5	5	10	15	20	25
4	4	8	12	16	20
3	3	6	9	12	15
2	2	4	6	8	10
1	1	2	3	4	5
Risk ↑ Impact →	**1**	**2**	**3**	**4**	**5**

Table 22: Assigning task priority in terms of risk and impact.

21 Software Engineering

21.1 The Software Development Lifecycle

This lecture was taught by Dr Nelly Bencomo.

21.1.1 Introduction to Software Development

Definition 21.1.1.1: The software development lifecycle is split into 6 stages: planning, analysis, design, implementation, testing and integration, and maintenance. This can be implemented in a number of ways:

- Waterfall: Planned, sequential stages. Good: Visible stage boundaries. Bad: Feedback can arrive late.
- V-Model: Connects development levels to verification and validation. Good: Explicit assurance. Bad: Changes can be costly.
- Incremental: Deliver useful functionality in successive parts. Good: Early value. Bad: Integration and architecture.
- Iterative: Refine a solution through repeated feedback. Good: Learning. Bad: Potential rework.
- Spiral: Cycles driven by risk assessment and reduction. Good: Tackles high risks early. Bad: Expertise and overhead.
- Agile: Adaptive planning and frequent valuable increments. Good: Responsiveness. Bad: Sustained collaboration required.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

21.1.2 Agile

Definition 21.1.2.1: The Agile Manifesto (first used in 2001) defines their priorities as people (individuals and interactions over processes and tools), software (working software over comprehensive documentation), collaboration (customer collaboration over contract negotiation), and change (responding to change over following a plan).

It requires engineering discipline, implementing requirements (clarify and revisit user needs), design (make and review architecture decisions), quality (test and integrate continuously), and feedback (inspect results with stakeholders).

Definition 21.1.2.2: A scrum is a set period of time within an project and involves a product owner, scrum master, and development team.

1. When beginning a scrum, the product owner will usually provide a product backlog which is taken into a sprint planning meeting.
2. Here, the scrum master discusses with the development team and sets out the work to be completed during the upcoming sprint.
3. The team then start a sprint and work through as much of the backlog as possible.
4. At the end of a chosen period of time (usually 1-4 weeks), the team carry a sprint review, identifying work done and issues preventing progress, before potentially choosing to release a product increment.
5. Ideally there should also be regular checkups throughout the sprint to identify progress and potential issues.

21.2 Agile Development in Practice

This lecture was taught by Dr Nelly Bencomo.

21.2.1 Approaches to Programming

Definition 21.2.1.1: Extreme Programming (XP) is built from five core values: communication, simplicity, feedback, courage, and respect. Whilst doing this, we take engineering discipline into account by making small releases early to get client feedback, making continuous integrations to reduce surprises when merging teammates' work, and refactoring frequently to keep change affordable.

Generally, we split XP into 12 basic practices: the planning game, small releases, metaphor, simple design, test-first programming, refactoring, pair programming, continuous integration, collective ownership, on-site customer, 40-hour weeks, and an open workspace.

Definition 21.2.1.2: Pair programming is the practice of coding in pairs where one person is the driver and uses the keyboard, while the other is the observer who discusses with the driver and ensures the code makes sense. The pair swap roles on a regular basis. This provides learning benefits for less-experienced programmers.

Definition 21.2.1.3: Test-first development is the practice of specifying an observable test, implementing the smallest change that passes the test before running all other existing tests and then refactoring the solution until all other tests are green.

21.2.2 Task Management

Definition 21.2.2.1: Agile projects generally utilise a Kanban board to keep track of project progress. It consists of columns representing the workflow stages (to-do, in progress, review, done, etc.) and cards representing individual tasks. The "in progress" column is often limited to a set number of items to reduce the likelihood of burnout.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

21.3 Project Requirements

21.3.1 Requirements I

This lecture was taught by Professor Pete Sawyer, taking content from his book: Requirement Engineering: A Good Practice Guide.

Requirements are important for a project as having wrong requirements can lead to the resulting software not being fit for purpose and errors can propogate and become costly to fix later. It is important to validate early to ensure you have the right requirements. As such, requirements statements become about communication. Stakeholders need confidence you understand what they want and developers need to know what to build.

Definition 21.3.1.1: Functional requirements give details about features the software provides.

Definition 21.3.1.2: Non-functional requirements give details about qualities or constraints for the project. They can be dynamic (reliability, accuracy, availability, etc.) or static (maintainability, compliance, etc.). They often act as design constraints such as budget or time.

Definition 21.3.1.3: People or organisations with requirements are stakeholders. Users perform the taret tasks, clients/customers pay for development, external/regulatory bodies ensure compliance, and other systems provide interaction requirements. As such, it is good practice to identify and document the software's stakeholders.

When trying to elicit requirements from stakeholders you can conduct interviews, observations, workshops, or focus groups. You can also add requirements after the software has deployed through bug reports and feature requests. UML models can also be useful to contextualise workflows and the organisation's architecture / stakeholders' requirements.

Verification ensures that requirements have been met. To do this, each requirement should have a test case. This can be challenging for NFRs as they can easily become poorly defined.

21.3.2 Requirements II

This lecture was taught by Dr Nelly Bencomo.

Definition 21.3.2.1: A software requirement is a functional or non-functional need to be implemented in the system.

- A functional requirement could provide a particular service to the user.
- A non-functional requirement can be a performance requirement.

Definition 21.3.2.2: Non-functional requirements are characteristics of the system which cannot be expressed as functions (e.g., maintainability, portability, security, etc.). These characteristics can be measurable (i.e., response time must be under 10 ms 95% of the time). Constraints are NFR (e.g., hardware to be used, OS, DBMS, etc.). Project management issues are also NFR (e.g., costs, schedules, logistics, etc.).

Definition 21.3.2.3: When gathering requirements, we can use the MoSCoW prioritisation method where requirements are sorted into one of "must haves", "should haves", "could haves", and "won't haves". We can then designate our workflow accordingly, giving maximum 60%, 20%, 20%, and 0% effort to these areas accordingly.

Definition 21.3.2.4: Behaviour driven development focuses on user stories and supports and encourages collaborative development. It is built on test-driven development and acceptance test driven development.

Definition 21.3.2.5: When presenting a requirement back to the user it should be human-readable by non-experts whilst still explaining the proposed workflow. Therefore, we use Gherkin which is the syntax language of Cucumber. This description is behavioural in nature but leads to a clearer understanding of what to code.

When outlining project requirements, we follow the following stages:

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

1. User needs domain information, existing system information, regulations, standards, etc.
2. Requirements Elicitation.
3. Requirements Analysis and Negotiation.
4. Requirements Documentation.
5. Requirements Validation (and optionally dropping back to any previous stage if needed).
6. This approach produces a requirement document and system specification which can then be approved by the client.

21.3.3 Requirements III

Definition 21.3.3.1: The role of requirements validation is to check that the right product is being built. It ensures that the software being developed (or changed) will satisfy the stakeholders and compares the software requirements specification against the stakeholders' goals and requirements. This is important as fixing a requirements error after delivery is much more expensive than fixing an implementation error.

To carry out this process, we use the following three steps throughout the process:

- *Elicitation*: Checking back with the elicitation sources (e.g., "So, are you saying that...?").
- *Analysis*: Checking that the domain description and requirements are correct.
- *Specification*: Checking that the defined system requirement will meet the user requirements under the assumptions of the domain/environment. Checking conformity to the well-formedness rules/standards.

Definition 21.3.3.2: The role of requirements verification is to check that the product is being built correctly. It ensures that each step followed in the process of building software yields the right products and checks the consistency of the software requirements specification artefacts and other software development products against the specification.

To carry out this process, we use the following steps throughout the process:

- *Validity Checks*: Does the system provide the functions which best support the customer's needs?
- *Consistency Checks*: Are there any requirements conflicts?
- *Completeness Checks*: Are all functions required by the customer included?
- *Realism Checks*: Can the requirements be implemented given available budget and technology?
- *Verifibility Checks*: Can the requirements be checked/tested?
- *Traceable*: Where/who did the requirement come from and rationale?

Definition 21.3.3.3: Each requirement carries a fit criteria. This is a benchmark or goal that the testing activity uses to determine whether the eventual solution satisfies the requirement. For functional requirements (FR), the function is either completed or not, and thus has no scales of measure. A non-functional requirement (NFR) is a quality the product must have, and fit criterion is a measure of that.

If such criteria cannot be easily expressed or quantified, then the requirement is likely to be ambiguous, incomplete, or incorrect. It is relatively easy to determine fit criteria for quality requirements that are naturally quantitative.

Definition 21.3.3.4: The prototyping process can start with those requirements which are poorly understood. This helps stakeholders understand what the system will do.

Prototypes for validation must be more complete than for elicitation as they must contain sufficient facilities for practical use to be made of it. This is because implementation of a number of features can help the user(s) to identify important features and those that are no so important.

We should translate requirements into a testable specification using the following process: understanding the client's need; creating a user story; setting fit criteria; writing Gherkin code to represent it; and then setting priority and rationale.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

22 Legal, Social, Ethical, and Professional Issues

This recommended reading for this sub-topic is:

- The Software Engineering Code of Ethics and Professional Practice, ACM (2024).

22.1 Introduction to Software Engineering and Software Engineering Ethics

This lecture was taught by Dr Craig Stewart.

22.1.1 Introduction to Software Engineering

Definition 22.1.1.1: An ill-structured problem, as defined by (Simon, 1973), is: elusive, unbounded, not understandable, without end conditions. They require orientation and context, and have many stakeholders and many possible solutions.

Definition 22.1.1.2: A good problem solver needs the following:

- **Method**: Refers to a formal procedure for accomplishing a goal, typically independent of the tools used.
- **Tool**: An instrument or automated system for accomplishing something in a better way.
- **Procedure**: A combination of tools and techniques to produce a product.
- **Paradigm**: The philosophy or approach for building a project.

Definition 22.1.1.3: For this module, a stakeholder will generally fall into one of three roles:

- **Customer/Client**: The company, organisation, or person who pays for the software system.
- **Developer**: The company, organisation, or person who is building the software system.
- **User**: The person/people who will actually use the system.

22.1.2 Introduction to Ethics

Definition 22.1.2.1: The software engineering code of ethics is defined by Rost and Glass (2010) and summarised more thoroughly by the Association for Computing Machinery (2024).

23 Group Project

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

Part VI
Theory of Computation

24 Module Introduction

Theory of Computation has the module code COMP2181 and is led by Dr Barnaby Martin.

24.1 Assessment

This module is assessed through two summative benchtests, and one final exams, these can be seen in Table 23. Additional formatives are also shown in italics.

Title	Opening	Due	Feedback	Professor	Weight
Benchtest 1	Week 11	-	2nd Feb	Dr Dantchev and Dr Mertzios	12.5%
Benchtest 2	Week 20	-	-	Dr Dantchev and Dr Mertzios	12.5%
Final Exam	May Exam	-	-	Dr Barnaby Martin	75%

Table 23: Assessments for Theory of Computation.

24.2 Timetable

This module is taught through two one-hour lectures and one two-hour computer class each week as seen below in Table 24.

Day	Time	Class Type	Location	Lecturer
Tuesday	10:00 - 11:00	Lecture	CLC202	Dr Barnaby Martin
Wednesday	09:00 - 11:00	Problem Class	TLC116	-
Thursday	14:00 - 15:00	Lecture	CLC202	Dr Sukanya Pandey

Table 24: Michaelmas Term Timetable for Theory of Computation.

25 Models of Computation

This section is taught by Dr Barnaby Martin. Check for recommended reading.

25.1 Finite-State Automata and Regular Languages

25.1.1 Formal Definitions

Definition 25.1.1.1: A deterministic finite-state automaton is a five-tuple $(Q, \Sigma, \delta, q_0, F)$ where Q is the finite set of states, Σ is a finite alphabet, $\delta : Q \times \Sigma \to Q$ is the transition function, $q_0 \in Q$ is the start state, and $F \subseteq Q$ is the set of accept states.

Definition 25.1.1.2: A DFA M recognises the language L if $L = \{w | M \text{ accepts } W\}$.

Definition 25.1.1.3: A language is called a regular language if some DFA recognises it.

Definition 25.1.1.4: A regular expression R defines a language $L(R)$. We shall eventually prove that $RE \equiv DFA$. R is a regular expression over the alphabet Σ if R is:

1. a for some $a \in \Sigma$.
2. ϵ (the empty string).
3. $\emptyset$ (the empty set).

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

4. $(R_1 \cup R_2)$ where R_1 and R_2 are regular expressions.
5. $R_1 \cdot R_2$ where R_1 and R_2 are regular expressions.
6. R_1^* where R_1 is a regular expression.

The list of regular operations can be seen in Table 25 where the operations labelled with $^+$ are language theory specific.

Operation Name	Notation	Set Equivalent
Union	$A \cup B$	$\{x \mid x \in A \text{ or } x \in B\}$
Intersection	$A \cap B$	$\{x \mid x \in A \text{ and } x \in B\}$
Difference	$A \backslash B$	$\{x \mid x \in A \text{ and } x \notin B\}$
Complement	$\overline{A}$	$\Sigma * \backslash A$
Concatenation$^+$	$A \circ B$	$\{xy \mid x \in A \text{ and } y \in B\}$
Star$^+$	$A*$	$\{x_1x_2...x_k \mid k \geq 0 \text{ and } x_i \in A \text{ for every } i, 1 \leq i \leq k\} \equiv \cup_{i=0}^{\infty} A^i$

Table 25: Regular Operations.

25.1.2 Combining Automata

Sometimes we may want to implement an operation on multiple languages. We can do this by combining their automata. If we have that L_1 is recognised by $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ and L_2 is recognised by $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$; we want to combine M_1 and M_2 into a new automaton M that would recognise $L_1 \cup L_2$.

You would think that we could just run the automata sequentially, and accept if either M_1 or M_2 accept, but this does not work. This is because after M_1 has run on the input, the input is exhausted and there is no way to rewind it in order to run M_2.

Instead, we can attempt to run M_1 and M_2 on the input in parallel. We do this by using pairs of states (one for each DFA). Formally, the components of the new automaton $M = (Q, \Sigma, \delta, q_0, F)$ are:

1. The set of possible states contains every possible cross-set pair of Q_1 and Q_2.

$$Q = Q_1 \times Q_2$$

2. For every $s \in Q_1, u \in Q_2, a \in \Sigma$. The transition function contains every transition from a pair of states from Q being given any input from Σ.

$$\delta((s, u), a) = (\delta_1(s, a), \delta(u, a))$$

3. The start state is the pair containing both start states from M_1 and M_2.

$$q_0 = (q_1, q_2)$$

4. The set of accept states is every possible cross-set pair of accept states from F_1 and F_2.

$$F = \{(s, u) \in Q \mid s \in F_1 \text{ or } u \in F_2\}$$

25.1.3 Non-Deterministic Finite Automaton

If, instead you wish to work with concatenation, such that $w \in L_1 \circ L_2$ only if there are words x and y such that $w = xy$, $x \in L_1$ and $y \in L_2$. You need to run M_1 on a prefix of w and then run M_2 on the rest. This must be done non-deterministically and stops becoming a DFA as all accept states of M_1 have to map to the start state of M_2.

Definition 25.1.3.1: A non-deterministic finite automaton is a five-tuple $(Q, \Sigma, \delta, q_0, F)$, where Q is the finite set of states, Σ is a finite alphabet, $\delta : Q \times (\Sigma \cup \{\epsilon\}) \to \mathscr{P}(Q)$ is the transition function, $q_0 \in Q$ is the start state, and $F \subseteq Q$ is the set of accept states.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

Computing NFA

Before computing an NFA, an important convention to note is that a single step in a computation consists of first reading the next input symbol followed by taking any number of ϵ-transitions. This means that empty strings ϵs can be freely added inside the actual word $\in \Sigma*$, e.g., 010110 can be viewed as, say, $\epsilon 01\epsilon\epsilon 01\epsilon 01$.

Let $M = (Q, \Sigma, \delta, q_0, F)$ be an NFA and $w = w_1 w_2 ... w_n$ be a word over $\Sigma \cup \{\epsilon\}$. M accepts w if there exists a sequence of states $r_0, r_1, r_2, ..., r_n$ satisfying the following conditions:

1. $r_0 = q_0$.
2. $r_{i+1} \in \delta(r_i, w_{i+1})$ for every $i, 0 \leq i \leq n-1$.
3. $r_n \in F$.

25.1.4 Converting NFA into DFA

The difference between a DFA and NFA is that a DFA has a unique computation (path) and either accepts or rejects, while an NFA generally has many computation paths and accepts if at least one computation accepts and only rejects if all the computations reject.

Theorem 25.1.4.1: Every NFA has an equivalent DFA.

Proof: To formally convert an NFA to a DFA, let $N = (Q, \Sigma, \delta, q_0, F)$ be an NFA recognising some language L (assume that N has no ϵ-transitions). We shall construct a DFA $M = (Q', \Sigma, \delta', q_0', F')$ that recognises the same language L as follows.

1. $Q' = \mathscr{P}(Q)$.
2. For $R \in Q'$ and $a \in \Sigma$, let

$$\delta'(R, a) = \{q \in Q | q \in \delta(r, a) \text{ for some } r \in R\} \tag{105}$$

   equivalently,

$$\delta'(R, a) = \bigcup_{r \in R} \delta(r, a) \tag{106}$$

3. $q_0' = \{q_0\}$.
4. $F' = \{R \in Q' | \exists r \in R, r \in F\}$.

Now, to consider ϵ-transitions, for $R \subseteq Q$, let

$$E(R) = \{q | q \text{ is reachable from } R \text{ through a number of } \epsilon\text{-transitions.}\} \tag{107}$$

Now, taking the above steps, we modify (105) to

$$\delta'(R, a) = \{q \in Q | q \in E(\delta(r, a)) \text{ for some } r \in R\} \tag{108}$$

and (106) to

$$\delta'(R, a) = \bigcup_{r \in R} E(\delta(r, a)) \tag{109}$$

We then modify step 3 to say $q_0' = E(\{q_0\})$.

□

Corollary 25.1.4.1: A language is regular iff some NFA recognises it.

Proof: See Theorem 25.1.4.1.

□

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

Theorem 25.1.4.2: A language is regular iff some regular expression describes it.

Proof: $\Leftarrow$ Given a regular expression R, construct a NFA that recognises $L(R)$ by structural induction:

- *Base Case*: All initial regular expressions can be implemented by NFAs.
- *Inductive Step*: We have already proven that the class of regular languages is closed under regular operations.

$\Rightarrow$ Given a DFA M, construct a regular expression that recognises $L(M)$.

1. Convert the given n-state DFA into an equivalent $n+2$ state GNFA.
2. Eliminate all the internal states of the GNFA one-by-one, while preserving the language recognised by the automaton.
   (a) Pick an internal state r for elimination.
   (b) For every two states u and v with transitions between them and r, $u \neq r, v \neq r$, combine the regular expressions of the three into one, new expression.
3. $L(R)$ should now equal $L(M)$.

□

Definition 25.1.4.1: A generalised non-deterministic finite automaton is defined as having a single start state with no incoming edges, a single end state with no outgoing edges, and every transition being labelled with a regular expression.

25.1.5 Non-Regular Expression

Definition 25.1.5.1: A non-regular expression is a language which cannot be recognised by any DFA. We prove a language is non-regular by assuming there is a DFA that recognises language L and then use an adversary argument against that DFA to get a contradiction.

Lemma 25.1.5.1: The pumping lemma states that, for every regular language L, there is a number p (the pumping length of L) such that every word $w \in L$ of length at least p can be divided into three parts $w = xyz$, satisfying the following conditions:

1. $xy^iz \in L$ for every $i \in N$ (in particular $xz \in L$ as one can take $i = 0$).
2. y is a non-empty string.
3. The length of the string xy is not greater than p.

Proof: Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA that recognises L, and let us set the pumping length p to be the number of states in $M : p = |Q|$. Consider the computation of M on the word $w = w_1w_2...w_n$ where $n \geq p$.

$$q_0 = r_0 \rightarrow w_1 \rightarrow r_1 \rightarrow w_2 \rightarrow ...r_{n-1} \rightarrow w_n \rightarrow r_n \in F \tag{110}$$

Let i be the first moment in time when we see a repetition in the sequence of states $r_0, r_1, ..., r_n$ (such an i must exist by the Pigeon-Hole principle, as $n+1 > p$). i.e., $r_i = r_j$ for some $i > j$ and the states $r_0, r_1, ..., r_{i-1}$ are all different. We can then set

$$x = w_1w_2...w_j \quad y = w_{j+1}...w_i \quad z = w_{i+1}...w_n \tag{111}$$

It is now straightforward to check that all three conditions are met.

□

Example 25.1.5.1: Prove that the language $L = \{0^n 1^n | n \in \mathbb{N}\}$ is not regular.

Proof: Take the word $0^p 1^p \in L$ where p is the pumping length of L. If $0^p 1^p = xyz$ and $|xy| \leq p$, then $x = 0^i, y = 1^j$ and $z = 0^{p-i-j}1^p$. But then $xy^2z = 0^{p+j}1 \notin L$ as $|y| > 0$, i.e., $j > 0$.

□

Definition 25.1.5.2: An algorithm that minimises any given DFA, i.e., finds an equivalent DFA with the minimal number of states. It answers both "What is the smallest DFA that recognises the same language?" and "Given two DFAs, do they recognise the same language?". It can be achieved by removing any states not reachable from the start state and contracting a set of states that are equivalent into a single state.

Definition 25.1.5.3: Two states s and t are equivalent if any word w, which is accepted when starting from s, is also accepted when starting from t.

Definition 25.1.5.4: A extended transition function is defined by letting a DFA $M = (Q, \Sigma, \delta, q_0, F)$ be given, and let $\hat{\delta} : Q \times \Sigma^* \to Q$ be defined as:

$$\hat{\delta}(s, \epsilon) = s \tag{112}$$

$$\hat{\delta}(s, w_1 w_2 ... w_n) = \hat{\delta}(\delta(s, w_1), w_2, ..., w_n) \tag{113}$$

for every state s and every word $w_1 w_2 ... w_n$.

25.2 Büchi Automata and Omega-Regular Languages

25.2.1 Finite-State Automata Over Infinite Words

Definition 25.2.1.1: If we have a NFA where the input is infinite (i.e., Σ^ω), we have a Büchi automaton. A Büchi automaton accepts $w_1 w_2 ... \in \Sigma^\omega$ if there is a sequence of states $r_0, r_1, r_2, ... \in Q^\omega$ satisfying the following conditions:

- $r_0 = q_0$.
- $(r_i, w_{i+1}, r_{i+1}) \in \Delta$ for every $i > 0$.
- There are infinitely many r_i's in F.

Definition 25.2.1.2: An ω-regular language/expression is build upon regular languages, using the following operations:

- $A \cup B$, where both A and B are ω-regular.
- AB, where A is regular, and B is ω-regular.
- A^ω, which is $\{a_1 ... | a_i \in A\}$, i.e., an infinite sequence of words from A, where A is regular and doesn't contain the empty word.

Let A be a regular language. The limit of A is the language $\{a \in \Sigma^\omega | a \text{ has infinitely many prefixes in } A\}$.

$L_1 = \{a \in \Sigma^\omega | a \text{ contains infinitely many 0s.}\}$.
$L_2 = \{a \in \Sigma^\omega | a \text{ contains finitely many 1s.}\}$.

Proposition 25.2.1.1: L_2 cannot be recognised by a deterministic Büchi automaton.

Proof: Assume that L_2 can be recognised by some deterministic Büchi automaton M. We will construct a word accepted by M that contains infinitely many 1s and thus doesn't belong to L_2.

1. $0^\omega \in L_2$ must be accepted by M, so there is some finite prefix 0^{n_1} that leads into accepting state p_1.
2. $0^{n_1}10^\omega \in L_2$ must be accepted by M, so there is some finite prefix $0^{n_1}10^{n_2}$ that leads into accepting state p_2.

3. $0^{n_1}10^{n_2}10^{\omega} \in L_2$ must be accepted by M, so there is some finite prefix $0^{n_1}10^{n_2}10^{n_3}$ that leads into accepting state p_3.

4. ...

The limit of the infinite process is a word of the form $(0^*1)^{\omega} \notin L_2$ that is accepted by M since it visits an infinite sequence of accepting states $p_1, p_2, ...$

□

25.2.2 Modal Logic

Definition 25.2.2.1: The box operator is used with propositional logic $\Box\varphi$ to mean "I know/can prove" φ. More generally - necessarily φ. Probably $\Box\varphi \to \varphi$, but not the other way round (φ is true, but we may not be able to prove it).

Definition 25.2.2.2: The diamond operator is used in $\neg\Box\varphi \stackrel{\text{def}}{=} \diamond\neg\varphi$ to indicate that you cannot prove φ, so it is possible that $\neg\varphi$. Conversely, $\neg \diamond \varphi$ (φ is impossible) is the same as proving $\neg\varphi$, i.e., $\Box\neg\varphi$. Simply, $\diamond$ means "it is possible that".

From these definitions and propositional logic, we get:

- $\frac{\varphi}{\Box\varphi}$ is the Neccesitation Rule **N**.
- $\Box(\varphi \to \psi) \to (\Box\varphi \to \Box\psi)$ is the Distribution Axiom **K**.

Definition 25.2.2.3: Kripke semantics state that, over a set of atomic propositions (Boolean variables) P, and a set of worlds (states) W, we are given a valuation function

$$f : P \times W \to \{true, false\} \tag{114}$$

and an accessibilty relation

$$R \subseteq W \times W \tag{115}$$

$(u, v) \in R$ if v is accessible from u. $\Box a$ holds at u iff a holds at every v, s.t. $(u, v) \in R$. Thus, $\diamond a$ holds at u iff a holds at some v, s.t. $(u, v) \in R$.

25.3 Linear-Time Logic and Translating Into Büchi Automata

26 Algorithms and Complexity

This section is taught by Dr Sukanya Pandey. The recommended reading for this topic is:

- Introduction to Algorithms (Cormen T.H., et al., 2009).

26.1 Asymptotic Complexity and Notation

Lecture one was just a recall of some of last year's Algorithms and Data Structures module.

26.2 Greedy Algorithms

Definition 26.2.0.1: The knapsack problem is defined as one of two variants. Either:

- The 0-1 knapsack problem (the classic variant), where a thief robs a store and finds n items. Item i is worth v_i euros and weighs w_i kilos. The thief can carry at most W kilos in his knapsack. *Which items can he take to maximise his profit?*

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

- the fractional knap problem, which has the same setup, but this time the thief can take fractions of items.
- We can assume (without loss of generality) that the knapsack is not big enough to carry all items:

$$W < w_1 + w_2 + ... + w_n \tag{116}$$

26.2.1 The Greedy Approach

To solve the fractional problem, the greedy approach would be to calculate the value per kilo using (117) for each item. You then sort the items by value per kilo, in decreasing order. You then take as much of the first item as possible, then the second item, etc., until you reach the capacity of the knapsack W.

$$r_i = \frac{v_i}{w_i} \tag{117}$$

Example 26.2.1.1: For example, if $W = 10$, and $\mathbf{v} = \{4, 10, 12, 20\}$ and $\mathbf{w} = \{4, 2, 8, 5\}$. We can then calculate $\mathbf{r} = \{1, 5, 1.5, 4\}$. The greedy algorithm then dictates that you should take all of item 2, all of item 4, and 3 kilos of item 3 $W = 2 + 5 + 3$. This gives a total value of $V = 10 + 20 + (3 \times 1.5) = 34.5$.

The running time of this algorithm can be calculated by taking that:

1. Step 1, calculating the value per kilo takes $\Theta(n)$.
2. Step 2, sorting the value per kilo array takes $\Theta(n\ log\ n)$.
3. Step 3, selecting the items can be done in $\Theta(n)$.

Therefore, the worst-case running time is $\Theta(n\ log\ n)$.

However, approach will not work for the 0-1 knapsack problem.

26.2.2 Correctness of the Greedy Algorithm for Fractional Knapsack

n items are sorted by value per kilo, (117) in decreasing order, i.e.,

$$r_1 \geq r_2 \geq ... \geq r_n \tag{118}$$

Two items, p and q with $r_p = r_q$ are considered to be equivalent. Hence, we may assume that

$$r_1 > r_2 > ... > r_n \tag{119}$$

The total available amount of item i is w_i. Let k be such that

$$w_1 + w_2 + ... + w_k \leq W \tag{120}$$

and

$$w_1 + w_2 + ... + w_{k+1} > W \tag{121}$$

Let $\mathbf{x} = (x_1, x_2, ..., x_n)$ denote a solution: $0 \leq x_i \leq w_i$ is the amount the thief takes of item i for $i = [0, n]$. The total value taken is then given by

$$\sum_{i=1}^{n} r_i x_i \tag{122}$$

The greedy solution $\mathbf{x}$ is given by $x_i = w_i$ for $1 \leq i \leq k$ and

$$x_{k+1} = W - (w_1 + w_2 + ... + w_k) \tag{123}$$

$x_i = 0$ for $k + 2 \leq i \leq n$. Let $\mathbf{x}^* = (x_1^*, x_2^*, ..., x_n^*)$ be an optimal solution.

Suppose that $\mathbf{x}^* \neq \mathbf{x}$. Then

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

- $x_i^* < x_i$ for some $i \leq k+1$.
- Let $j = \min\{i : x_i^* < x_i\}$.
- And $x_l^* > x_l$ for some $l > j$.

Let us add some more of item j to the optimal choice. Let y be another solution, defined by

$$y = \begin{cases} y_j = \min\{w_j, x_j^* + (x_l^* - x_l)\} \\ y_l = x_l^* - (y_j - x_j^*) \\ y_i = x_i^* \text{ otherwise.} \end{cases} \tag{124}$$

26.3 Divide-and-Conquer Algorithms

26.3.1 Colouring for Restricted Inputs

Definition 26.3.1.1: Many useful algorithms utilise recursions to quickly resolve subproblems. We refer to these as divide-and-conquer algorithms, as at each step they:

1. *Divide* the problem into a number of subproblems.
2. *Conquer* the subproblems by solving them, either recursively or directly depending on size.
3. *Combine* the solutions for the subproblems into a solution for the original problem.

A natural way to calculate the running time of these algorithms is recurrence relations using the Master Theorem (covered in Year 1 - Algorithms and Data Structures).

Definition 26.3.1.2: A colouring of a graph is an assignment of colours to its vertices such that any two adjacent vertices get a different colour. A colouring using at most k colours is a k-colouring.

Definition 26.3.1.3: The chromatic number, χ_G, of a graph is the smallest integer k for which G has a k-colouring. Calculating the chromatic number of any given graph is considered a NP-hard problem.

Definition 26.3.1.4: Let G_1 and G_2 be two vertex-disjoint graphs. The disjoint union $G_1 + G_2$ is the graph with vertex set $V(G_1) \cup V(G_2)$ and edge set $E(G_1) \cup E(G_2)$.

Definition 26.3.1.5: Let G_1 and G_2 be two vertex-disjoint graphs. The join $G_1 \times G_2$ is the graph with vertex set $V(G_1) \cup V(G_2)$ and edge set $E(G_1) \cup E(G_2) \cup E^*$, where E^* is the set of all edges between vertices of G_1 and vertices of G_2.

Definition 26.3.1.6: Let G_1 and G_2 be two vertex-disjoint graphs. A graph G is a cograph if it can be created using the following rules:

- The 1-vertex graph is a cograph.
- If G_1 and G_2 are cographs, then so is $G_1 + G_2$.
- If G_1 and G_2 are cographs, then so is $G_1 \times G_2$.

Alternatively, let P_4 denote the path on 4 vertices. A graph G contains P_4 as an induced subgraph if G can be modified to P_4 via vertex deletions. If not, then G is said to be P_4-free.

Theorem 26.3.1.1: A graph G is a cograph $\iff$ G is P_4-free.

Proof: Proof excluded.

□

Definition 26.3.1.7: Let G be a cograph. The cotree T_G of G is the unique decomposition tree that satisfies:

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

1. The root r of the tree represents the graph $G_r = G$.
2. Every leaf x of the tree represents exactly one vertex of G, and vice-versa.
3. Every internal node x of the tree has at least two children and is either labelled + or $\times$. The node represents an induced subgraph G_x of G, defined as follows:
   - If x is a +-node, then G_x is the disjoint union of all graphs G_y where y is a child of x.
   - If x is a $\times$-node, then G_x is the join of all graphs G_y where y is a child of x.
4. Along the path from the root r to any leaf, the labels of the internal notes alternate between + and $\times$.

Theorem 26.3.1.2: Any cograph G has a unique cotree. However, if we drop condition 4, there may be many trees representing G.

Proof: Proof excluded.

□

We can use a divide-and-conquer algorithm to colour a cograph. Given a cograph G, construct its cotree T_G. This can be done in $O(n+m)$ time (Corneil, Perl and Stewart, 1985). We follow a bottom-up approach: only process node x after first processing its children.

- *Leaves*: Each leaf represents a single vertex of G. The chromatic number of a single-vertex graph is 1.
- *Union-Nodes*: If x is a +-node, then χ_{G_x} is the maximum of the chromatic numbers of the graphs G_y, where y is a child of x.
- *Join-Nodes*: If x is a $\times$-node, then χ_{G_x} is the sum of the chromatic numbers of the graphs G_y, where y is a child of x.

This way, the last node we process is the root r, which represents the graph $G_r = G$. As the number of nodes of T is $O(n)$ and we used $O(n)$ time per node, the total running time is polynomial.

Algorithm 1 MAX-SUBARR

Input: A, low, high
Output: low, high, sum

if high = low **then return** (low, high, A[low])
else
  median $\leftarrow \left\lfloor \frac{\text{low+high}}{2} \right\rfloor$
  $(L_{low}, L_{high}, L_{sum}) \leftarrow$ MAX-SUBARR(A, low, median)
  $(R_{low}, R_{high}, R_{sum}) \leftarrow$ MAX-SUBARR(A, median+1, high)
  $(\text{Cross}_{low}, \text{Cross}_{high}, \text{Cross}_{sum}) \leftarrow$ MAX-CROSS-SUBARR(A, low, median, high)
  if $L_{sum} \geq \max\{R_{sum}, \text{Cross}_{sum}\}$ **then return** $(L_{low}, L_{high}, L_{sum})$
  end if
  if $R_{sum} \geq \max\{L_{sum}, \text{Cross}_{sum}\}$ **then return** $(R_{low}, R_{high}, R_{sum})$
  else return $(\text{Cross}_{low}, \text{Cross}_{high}, \text{Cross}_{sum})$
  end if
end if

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

Algorithm 2 MAX-CROSS-SUBARR

Input: A, low, mid, high
Output: max_left, max_right, left_sum+right_sum

```
left_sum← −∞
for i ←mid to low do
    sum ← sum+A[i]
    if sum > left_sum then
        left_sum ← sum
        max_left ← i
    end if
end for
for j ←mid+1 to high do
    sum ← sum+A[j]
    if sum > right_sum then
        right_sum ← sum
        max_right ← j
    end if
end for
return (max_left, max_right, left_sum+right_sum)
```

26.3.2 The Master Theorem

Definition 26.3.2.1: The master theorem is formally defined as follows:
Let $T(n)$ be a function on the natural numbers defined by

$$T(n) = aT\left(\frac{n}{b}\right) + f(n) \tag{125}$$

for some constants $a \geq 1$ and $b > 1$. Then:

1. If $f(n) = O\left(n^{\log_b a - \epsilon}\right)$ for some constant $\epsilon > 0$, then $T(n) = \Theta\left(n^{\log_b a}\right)$.
2. If $f(n) = \Theta\left(n^{\log_b a}\right)$, then $T(n) = \Theta\left(n^{\log_b a} \cdot \log n\right)$.
3. If $f(n) = \Omega\left(n^{\log_b a + \epsilon}\right)$ for some constant $\epsilon > 0$, and if $a \cdot f\left(\frac{n}{b}\right) \leq cf(n)$ for some constant $c < 1$ and for sufficiently large n, then $T(n) = \Theta(f(n))$.

When multiplying two matrices, $A, B \in \mathbb{R}^{n \times n}$ with output $C \in \mathbb{R}^{n \times n}$, we can normally complete the computation in time $\Theta\left(n^3\right)$ by using the following definition:

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj} \tag{126}$$

However, we can design a divide-and-conquer algorithm to do this much quicker using MULT(A,B). The running time of MULT(A, B) satisfies

$$T(n) = 8T\left(\frac{n}{2}\right) + \Theta\left(n^2\right) \tag{127}$$

since we have 8 multiplications of $\frac{n}{2} \times \frac{n}{2}$ matrices and 4 matrix additions, each taking time $\Theta\left(\frac{n}{2} \cdot \frac{n}{2}\right) = \Theta\left(n^2\right)$. Therefore, the master theorem implies that $T(n) = \Theta\left(n^3\right)$.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

Algorithm 3 MULT(A,B)

Input: $A, B \in \mathbb{R}^{n \times n}$
Output: $C \in \mathbb{R}^{n \times n}$

Let C be a new $n \times n$ matrix.
if $n = 1$ **then**
  $c_{1,1} \leftarrow a_{1,1} b_{1,1}$
else
  Partition A, B, C into the block matrices.
  $C_{1,1} \leftarrow \text{MULT}(A_{1,1}, B_{1,1}) + \text{MULT}(A_{1,2}, B_{2,1})$
  $C_{1,2} \leftarrow \text{MULT}(A_{1,1}, B_{1,2}) + \text{MULT}(A_{1,2}, B_{2,2})$
  $C_{2,1} \leftarrow \text{MULT}(A_{2,1}, B_{1,1}) + \text{MULT}(A_{2,2}, B_{2,1})$
  $C_{2,2} \leftarrow \text{MULT}(A_{2,1}, B_{1,2}) + \text{MULT}(A_{2,2}, B_{2,2})$
end if
return C

26.3.3 Strassen's Algorithm

Definition 26.3.3.1: Strassen (1969) made a sophisticated divide-and-conquer approach called Strassen's algorithm. It has four steps:

1. Divide A, B, and C into four parts as before, with running time $\Theta(1)$.
2. Create 10 auxiliary matrices $S_1, ..., S_10 \in \mathbb{R}^{\frac{n}{2} \times \frac{n}{2}}$, obtained by sums or differences of $A_{1,1}, ..., B_{2,2}$, with running time $\Theta\left(n^2\right)$.
3. Using all the matrices available, recursively compute seven product matrices $P_1, ..., P_7 \in \mathbb{R}^{\frac{n}{2} \times \frac{n}{2}}$, with running time $7T\left(\frac{n}{2}\right)$.
4. Compute $C_{1,1}, ..., C_{2,2}$ by adding/subtracting combinations of $P_1, ..., P_7$, with running time $\Theta\left(n^2\right)$.

Thus, the worst-case running time of Strassen's algorithm satisfies

$$T(n) \leq 7T\left(\frac{n}{2}\right) + \Theta\left(n^2\right) \tag{128}$$

so, by the master theorem, we obtain

$$T(n) = \Theta\left(n^{log_2 7}\right) = \Theta\left(n^{2.8074}\right) \tag{129}$$

26.3.4 Rod Cutting

Definition 26.3.4.1: The rod cutting problem states that you work for a company that buys steel rods and cuts them into shorter rods, which it then sells. You are asked to determine the best way to cut the rods so as to maximise profit. You know, for $i = 1, ..., n$, the price p_i in euros that the company charges for a rod of length i centimeters. Given a rod of length n and a table of prices p_i for $i = 1, ..., n$, determine the maximum revenue r_n obtained by cutting up the rod and selling the pieces.

Definition 26.3.4.2: An integer partition of a positive integer n is a list of positive integers $(a_1, ..., a_k)$ such that $a_1 \leq ... \leq a_k$ and

$$\sum_{i=1}^{k} a_i = n \tag{130}$$

Let $p(n)$ denote the number of integer partitions of n, then

$$p(1) = 1, p(2) = 2, p(3) = 3, p(4) = 5 \tag{131}$$

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

Hardy and Ramanujan proved that

$$p(n) \sim \frac{1}{4n\sqrt{3}} \exp\left(\pi\sqrt{\frac{2n}{3}}\right) \tag{132}$$

Therefore, brute force has an exponential running time.

Definition 26.3.4.3: Optimal Substructure: Optimal solutions to a problem incorporate optimal solutions to related subproblems, which we may solve independently.

More specifically, for the rod cutting problem, we have for all $n \geq 1$,

$$r_n = \max\{p_i + r_{n-i} : 1 \leq i \leq n\} \tag{133}$$

The idea behind this is that p_n corresponds to no cuts and $p_i + p_{n-i}$ to cutting into a rod of length i and another rod of length $n - i$. To solve the original problem of size n, we solve independent smaller problems of the same type, but of smaller sizes. For the rod problem, we can use the following algorithm:

Algorithm 4 ROD(p, n)

Input: p, n
Output: q

if $n = 0$ **then return** 0
end if
$q \leftarrow -1$
for $i \leftarrow 1$ to n **do**
    $q \leftarrow \max\{q, p[i] + ROD(p, n - i)\}$
end for
return q

26.4 Dynamic Programming Algorithms

26.5 Machinings in Graphs

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

Part VII
Appendix

27 Artificial Intelligence

27.1 Module Professors

Module Lead: Doctor Yang Long

- Focus: Scalable AI solutions for large-scale healthcare applications.
- Email: yang.long@durham.ac.uk
- Office Hours: Monday 15:30 - 16:30 in Term 1.

Lecturer: Professor Iain Stewart

- Focus: Computational complexity, graph/group theory, and theoretical aspects artificial intelligence.
- Email: i.a.stewart@durham.ac.uk
- Office Hours: Friday 14:00 - 15:00 in Term 2.

Lecturer: Doctor Shauna Concannon

- Focus: Computational linguistics, social interaction, and human-computer interaction.
- Email: shauna.j.concannon@durham.ac.uk
- Office Hours: Thursday 14:30 - 15:30 in Term 2.

27.2 References

27.3 List of Figures

27.4 Reading List

27.4.1 Machine Learning

Long, Y. and Durham University (2026). COMP2261_Machine_Learning_Study_and_Coursework_Guide-_with_Navigation.pdf.

Deisenroth, M.P., Faisal, A.A. and Ong, C.S. (2020). Mathematics for Machine Learning. [online] Cambridge University Press. Available at: https://mml-book.com/ [Accessed 3 Oct. 2026].

Hackeling, G. (2017). Mastering Machine Learning with scikit-learn. 2nd edn. Packt Publishing.

27.4.2 Ethics and Bias in AI

Various, will be added over time.

28 Data Science

28.1 Module Professors

Module Lead: Professor Thomas Erlebach

- Focus: Algorithms and networks.
- Email: thomas.erlebach@durham.ac.uk
- Office Hours: n/a.

Lecturer: Doctor Igor Razgon

- Focus: Algorithms and complexity, graph theory, and discrete mathematics.
- Email: igor.razgon@durham.ac.uk
- Office Hours: Friday 14:00 - 15:00.

Lecturer: Doctor George Alex Koulieris

- Focus: Imagr processing and computer graphics.
- Email: georgios.a.koulieris@durham.ac.uk
- Office Hours: Please email to make an appointment.

28.2 References

28.3 List of Figures

28.4 Reading List

28.4.1 Probability

Degroot, M.H. and Schervish, M.J. (2012). Probability and Statistics. Boston Mass.: Pearson.

28.4.2 Data Cleaning and Analytics

McKinney, W. (2022). Python for Data Analysis. O'Reilly Media, Inc.

29 Networks and Systems

29.1 Module Professors

Module Lead: Doctor Wanqing Tu

- Focus: Computer networks and wireless communications.
- Email: wanqing.tu@durham.ac.uk
- Office Hours: Thursday 11:00 - 12:00.

Lecturer: Doctor Thirupathaiah Vasantam

- Focus: Applied probability and performance modelling.

- Email: thirupathaiah.vasantam@durham.ac.uk
- Office Hours: Monday 14:00 - 15:00.

Lecturer: Doctor Carlton Shepherd

- Focus: Cyber security.
- Email: carlton.g.shepherd@durham.ac.uk
- Office Hours: Email to make an appointment.

29.2 References

29.3 List of Figures

29.4 Reading List

29.4.1 Cyber Security

Nakov, S., Stefanov, M. and Shideroff, M. (2018). Practical Cryptography for Developers. [online] sec. Encryption: Symmetric and Asymmetric Available at: https://cryptobook.nakov.com/ [Accessed 9 Oct. 2026].

Department of Computer Science, Durham University (2026). The Threshold Trade-Off. [online] Available on Durham University's Blackboard [Accessed 26 Oct. 2026].

29.4.2 Databases

Connolly, T.M. and Begg, C.E. (2015). Database Systems: A practical approach to design, implementation, and management. 6th edn. Harlow, Essex, England: Pearson Education Limited.

29.4.3 Distributed Systems

Van Steen, M. and Tanenbaum, A.S. (2025). Distributed Systems. 4th edn. [online] Self-Published. Available at: https://www.distributed-systems.net/index.php/books/ds4/ [Accessed 5 Oct. 2026].

30 Programming Paradigms

30.1 Module Professors

Module Lead: Doctor Stuart James

- Focus: Artificial intelligence, computer vision, human computer interaction, and digital humanities.
- Email: stuart.a.james@durham.ac.uk
- Office Hours: Email to make an appointment.

Lecturer: Doctor Billy Moses Jr.

- Focus: Algorithms and biologically inspired computing.
- Email: william.k.moses-jr@durham.ac.uk
- Office Hours: Email to make an appointment.

Lecturer: Doctor Mladen Ivkcovic

- Focus: Performance prediction and optimisation.
- Email: mladen.ivkovic@durham.ac.uk
- Office Hours: Email to make an appointment.

30.2 References

30.3 List of Figures

30.4 Reading List

30.4.1 Systems Programming

Bryant, R. and O'Hallaron, D. (2015). Computer systems: A Programmer's Perspective. 3rd edn. Pearson Education, Limited.

31 Software Engineering

31.1 Module Professors

Module Lead: Doctor Craig Stewart

- Focus: Adaptive systems, HCI, and user modelling.
- Email: craig.d.stewart@durham.ac.uk
- Office Hours: Friday 11:00 - 12:00.

Lecturer: Doctor Nelly Bencomo

- Focus: Software engineering for autonomous, self-adaptive, AI and ML systems.
- Email: nelly.bencomo@durham.ac.uk
- Office Hours: Email to make an appointment.

Lecturer: Doctor Gagangeet Aujla

- Focus: Data-driven and data-intensive cyber-physical systems and digital twins.
- Email: gagangeet.s.aujla@durham.ac.uk
- Office Hours: Email to make an appointment.

Lecturer: Doctor Gokberk Kabacaoglu

- Focus: Machine learning and numerical applications.
- Email: gokberk.kabacaoglu@durham.ac.uk
- Office Hours: Thursday 14:00 - 15:00.

Lecturer: Doctor Ryan Crosby

- Focus: The assessment of novice programmers.
- Email: ryan.crosby@durham.ac.uk

- Office Hours: Email to make an appointment.

Lecturer: Doctor Sukanya Pandey

- Focus: Graph theory.
- Email: sukanya.pandey@durham.ac.uk
- Office Hours: Email to make an appointment.

31.1.1 Guest Speakers

Guest Speaker: Professor Pete Sawyer

- Focus: Requirements engineering.
- Email: peter.h.sawyer@durham.ac.uk
- His Book: Requirements Engineering: A Good Practice Guide.

31.2 References

31.2.1 Legal, Social, Ethical, and Professional Issues

Simon, H.A. (1973). The Structure of Ill Structured Problems. Artificial Intelligence, 4(3-4).

Rost, J. and Glass, R.L. (2010). The Dark Side of Software Engineering: Evil on Computing Projects. Washington, DC: IEEE Computer Society.

Sommerville, I. and Sawyer, P. (1997). Requirements Engineering: A Good Practice Guide. John Wiley & Sons, Inc.

31.3 List of Figures

31.4 Reading List

31.4.1 Legal, Social, Ethical, and Professional Issues

Association for Computing Machinery (2024). The Software Engineering Code of Ethics and Professional Practice. [online] acm.org. Available at: https://www.acm.org/code-of-ethics/software-engineering-code [Accessed 7 Oct. 2026].

32 Theory of Computation

32.1 Module Professors

Module Lead: Doctor Barnaby Martin

- Focus: Complexity theory, finite model theory, and the links between logic and complexity.
- Email: barnaby.d.martin@durham.ac.uk
- Office Hours: Flexible, contact to arrange.

Lecturer: Doctor Sukanya Pandey

- Focus: Graph theory.

- Email: sukanya.pandey@durham.ac.uk
- Office Hours: Flexible, contact to arrange.

Lecturer: Doctor Stefan Dantchev

- Focus: Combinatorics.
- Email: s.s.dantchev@durham.ac.uk
- Office Hours: To be confirmed.

Lecturer: Doctor George Mertzios

- Focus: Algorithms and networks.
- Email: george.mertzios@durham.ac.uk
- Office Hours: To be confirmed.

32.2 References

Corneil, D.G., Perl, Y., and Stewart, L.K. (1985). A Linear Recognition Algorithm for Cographs. SIAM Journal on Computing, 14(4), pp.926-934. doi:10.1137/0214065.

32.3 List of Figures

32.4 Recommended Reading

32.4.1 Algorithms and Complexity

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2009). Introduction to Algorithms. 3rd edn. MIT Press.